



BACHELOR'S THESIS TEMPLATE - TITLE OF PROJECT

Bachelor's Thesis

Arthur Ernst Henrik Reuss, s5582253, a.e.h.reuss@student.rug.nl,

Supervisors: Gido van de Ven

Abstract: Replay-based continual learning is usually judged by the accuracy it reaches *after* each task. Continual evaluation has exposed a transient failure that this end-of-task view hides: the *stability gap*, a sharp drop in past-task accuracy at the start of every new task, before the model recovers. The gap persists even when the optimiser has access to the exact joint loss over all tasks, so it is not a defect of the objective but of the optimisation trajectory. We argue that, in its dominant components, the gap is a symptom of a single structural feature of online continual learning: at each task boundary the loss the optimiser follows changes discontinuously, from the replay loss alone to the sum of replay and current-task losses. We decompose the gap into three additive contributors — a first-step magnitude asymmetry, a finite-buffer estimator variance, and a trajectory effect — and identify the discontinuity as the source of the first and third. To remove it, we introduce the λ -*curriculum*, a homotopy that ramps the current-task loss weight from 0 to 1 over the first steps of each task, so the optimum moves along a continuous path rather than teleporting across the landscape. An envelope-theorem argument shows the replay loss is monotone non-decreasing along this path, bounding the trajectory contribution by $O(1/N)$ in the ramp length N . We add two refinements, a $\lambda_{\min}$ floor that restores early-task plasticity and momentum that suppresses the residual stochastic oscillation, and evaluate the method on rotated MNIST against vanilla and gradient-balanced Experience Replay.

Contents

1	Introduction	3
2	Background and Related Work	5
2.1	Continual Learning	5
2.2	Experience Replay	5
2.3	Path-Finding Methods	6
2.4	The Stability Gap	6
2.5	Blurry Task Boundaries	6
2.6	Linear Mode Connectivity	7
3	Motivating a Landscape-Shaping Approach	7
3.1	Decomposing the Stability Gap	7
3.2	Path-Finding versus Landscape-Shaping	8
4	The Envelope-Theorem Bound: Why a Linear λ-Ramp Works	8
4.1	Part A: The perfect path never overshoots	8
4.2	Part B: A slow ramp keeps the model on the path	9
4.3	The Combined Bound	9
5	Research Questions	10

6	Methods	10
6.1	Experimental Setup	11
6.2	Experiments	13
7	Results	18
7.1	Three-Contributor Decomposition (RQ1)	18
7.2	The λ -Curriculum on rot-MNIST (RQ2a, RQ2b, RQ2c)	19
7.3	Momentum Cross (RQ2d)	21
7.4	Generalisation: CIFAR-10 (RQ3)	22
7.5	Long-Sequence rot-MNIST	24
7.6	Summary of Findings	24
8	Discussion	24
8.1	The boundary is the cause, and it has two faces	26
8.2	Why shaping beats path-finding	26
8.3	Two mechanisms, two parts of the fix	27
8.4	Relation to blurry task boundaries	27
8.5	Limitations	27
8.6	Implications and outlook	28
A	Implementation Details	31
A.1	Datasets	31
A.2	Experience replay	31
A.3	Natural Continual Learning	31
B	Metric Definitions	32
C	Complete Results	34
C.1	rot-MNIST: Decomposition (G-series)	34
C.2	rot-MNIST: λ -Curriculum (C-series)	35
C.3	rot-MNIST: Buffer-Fidelity Diagnostics	35
C.4	CIFAR-10: Headline Block	35
C.5	CIFAR-10: Generalisation Block	35
C.6	Long-Sequence rot-MNIST (L-series)	36
D	Detailed Step-by-Step Proof of the Trajectory Bound	37

1 Introduction

Deep neural networks learn rich representations from static, independent, and identically distributed data, but a vulnerability appears as soon as training becomes sequential: the network overwrites previously acquired knowledge to accommodate new data. This phenomenon, catastrophic forgetting [19, 22], is a major hurdle in machine learning. While retraining from scratch solves the issue, it is simply too expensive for systems that need to adapt dynamically. Continual learning (CL) studies how to learn from a sequence of tasks without that cost, and its core difficulty is the stability–plasticity dilemma: the model must stay plastic enough to learn new distributions while staying stable enough to preserve old ones [20].

Several families of techniques address forgetting: replay, parameter and functional regularisation, optimisation-based methods, context-dependent processing, and template-based classification [25]. These methods reduce long-term forgetting, but they have historically been evaluated only at task boundaries or at coarse intervals, which hides what happens within a task. Evaluating the network continuously exposes a failure these protocols miss. Lange et al. [17] did exactly this and identified the *Stability Gap*, a transient but severe collapse in past-task accuracy that occurs immediately upon the introduction of a new task. Even a well-tuned Experience Replay model that finishes a task with high accuracy on earlier tasks passes through a deep minimum during the first hundred steps of the new task.

This matters for two reasons. Because deployed continual-learning systems are queried throughout training, a model caught mid-transition will commit errors that its final checkpoint never would. Furthermore, this temporary forgetting presents an interesting puzzle: the model clearly retains the knowledge needed to recover, meaning a specific mechanism must disrupt and later restore it. Forcing the model to relearn what it already knows wastes valuable compute, making it essential to understand this underlying mechanism so we can close the gap efficiently.

The discovery of the gap raised an immediate question: could it be removed by better approximating the joint loss over all tasks? Hess et al. [12] showed that the gap persists even under incremen-

tal joint training, where the model has access to all data from all tasks at every step. A perfect objective still produces transient forgetting, so the problem lies not in *what* is optimised but in *how* the optimisation proceeds.

To see why the trajectory itself produces the gap, consider the geometry of sequential optimisation. Figure 1.1 visualises a transition from a learned task (Task A) to a new task (Task B), a conceptual two-dimensional reduction of a high-dimensional surface that nonetheless captures the dynamics at play. Our depiction of the transition geometry borrows the loss-landscape lens of Kao et al. [14], who visualise sequential optimisation in this way to motivate curvature-aware updates; we adopt their geometric framing while drawing the opposite conclusion about how to respond to it (Section 3). The network starts at the converged minimum A of Task A and must reach the joint minimum $A \cap B$. Running stochastic gradient descent (SGD) on the joint loss from A follows the green curve: pulled by the steepest gradients of the joint landscape, the trajectory bows outward, leaving Task A’s basin and crossing a region of higher Task-A error before converging at $A \cap B$. The dip on Task A along this arc is the stability gap.

This trajectory is the consequence of a discontinuity at the task boundary. While Task A is learned the optimiser descends the replay loss alone, the red basin in Figure 1.1. The moment Task B begins, the objective switches in a single step to the sum of the replay and current-task losses, the purple joint basin. Its minimum at $A \cap B$ replaces the red one from one step to the next. The parameters do not move during this switch, yet the minimum they are chasing jumps from θ_A^* to θ_{joint}^* . We call this *landscape teleportation*: from the optimiser’s point of view the surface it was descending disappears and is replaced by another whose minimum lies elsewhere.

The teleportation does more than redirect the optimiser along a curved path; it also makes the very first step after the switch skewed. At θ_A^* the replay gradient is near zero, because the optimiser has converged on Task A, while the current-task gradient is large, because Task B is still unlearned. The summed update is therefore dominated by the new-task gradient, and SGD takes a long step out of Task A’s basin before the still-negligible replay term can pull it back. This magnitude imbalance

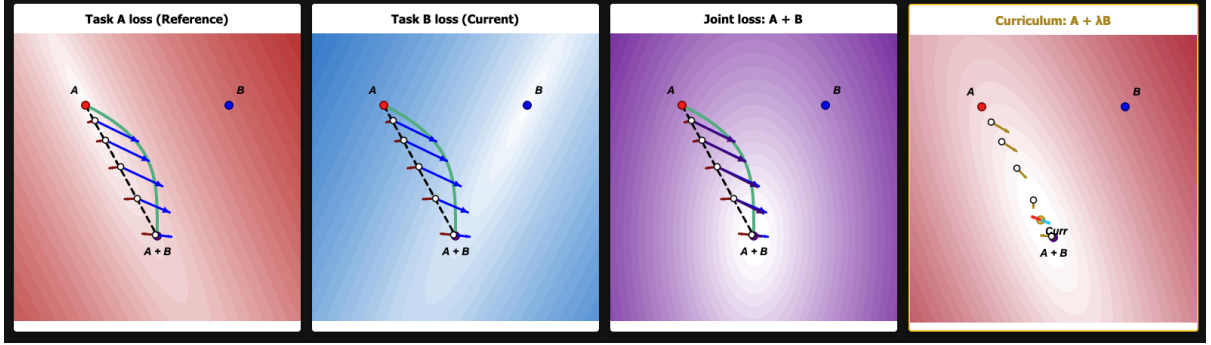


Figure 1.1: Conceptual visualisation of the optimisation landscape at a task transition, shown over a shared two-dimensional parameter space. Task A (red) and Task B (blue) are the single-task losses with minima at A and B ; the joint loss $A+B$ (purple) has its minimum at $A \cap B$. The dashed line is the straight interpolation from A to $A \cap B$; the green curve is the steepest-descent trajectory of the joint loss, which bows away from Task A’s basin and induces the stability gap. The fourth panel shows the λ -curriculum loss $A + \lambda B$: a small weight λ on Task B reshapes the basin into a valley that runs continuously from A toward the joint minimum, so the optimum moves along the corridor instead of teleporting across it. Arrows are normalised to a fixed length and show gradient direction only.

compounds the trajectory effect: not only does the descent path bow away from Task A’s minimum, the first step along it is the largest and the least opposed, which is why the dip is steepest in the first updates after a task switch [17]. A finite replay buffer adds a third, smaller error: it estimates the past-task gradient from a handful of stored samples rather than from all past data, so even the replay signal it supplies is noisy. Trajectory effect, magnitude asymmetry, and estimator variance are the three contributors to the gap; Section 3 makes their decomposition precise and the experiments measure each in isolation.

Existing methods take this discontinuity as given. They accept that the objective jumps at the task boundary and instead make the optimiser robust to the jump, constraining each SGD update with information about the past tasks, their gradients or local curvature, so that SGD avoids directions in which the replay loss rises sharply [3, 14, 15, 18]. We group these approaches under the term *path-finding*: while effective, they navigate the broken landscape rather than fixing it.

This thesis takes the opposite route. Rather than steer the optimiser carefully across a discontinuity, we propose removing the discontinuity itself, an approach we call *landscape-shaping*. We instantiate it as the λ -**curriculum**: instead of switching the ob-

jective from the replay loss to the joint loss in a single step, the curriculum ramps the current-task loss weight from 0 to 1 over the first N steps of each transition,

$$\begin{aligned} L_\lambda(t) &= L_{\text{replay}} + \lambda(t) L_{\text{current}}, \\ \lambda(t) &= \text{clip}\left(\frac{t}{N}, 0, 1\right), \end{aligned} \quad (1.1)$$

so the optimum $\Theta^*(\lambda)$ traces a continuous curve from θ_A^* (at $\lambda = 0$) to θ_{joint}^* (at $\lambda = 1$), and the model follows this moving valley instead of being teleported across the discontinuity (fourth panel of Figure 1.1). That such a continuous low-loss route from θ_A^* to θ_{joint}^* exists at all is what work on linear mode connectivity [5, 7] establishes. These studies show that distinct optima are often connected by simple paths of near-constant loss. We visualize this path as the dashed corridor in Figure 1.1. The curriculum’s role is to keep the optimiser on this specific route and prevent it from bowing away. Because the optimum now moves continuously, the curriculum addresses two of the three contributors at once. It removes the trajectory effect, since the descent no longer bows away from Task A’s basin (the green curve), and it tempers the magnitude imbalance, since the large new-task gradient is introduced gradually rather than dominating the first step. Only the third contributor, the variance of the finite-buffer estimator, is independent of the sched-

ule and left untouched. Treating the task-boundary discontinuity as the object to be fixed is the central contribution of this thesis; Section 4 makes the argument precise.

Beyond the basic linear ramp, we develop three refinements, each targeting a limitation of the schedule: an **adaptive schedule** that removes the need to choose the ramp length N , a $\lambda_{\min}$ **floor** that prevents the schedule from stalling progress on the new task, and **momentum** that suppresses the per-step accuracy oscillation the continuous-limit argument abstracts away. Section 6 describes how each is constructed and why.

The remainder of this thesis develops these ideas. Section 2 reviews continual learning, Experience Replay, the path-finding methods we compare against, the stability-gap literature, and two lines of work, blurry task boundaries and mode connectivity, that motivate the curriculum. Section 3 then sets out the thesis’s own account of the gap: it decomposes the gap into three contributors and contrasts path-finding with landscape-shaping. Section 4 proves that a continuous linear λ -ramp removes the trajectory contributor, with a residual of order $1/N$ in the ramp length. Section 5 states the research questions, and Section 6 describes the experimental testbed and the conditions used to test each prediction.

2 Background and Related Work

2.1 Continual Learning

To address catastrophic forgetting and the stability–plasticity dilemma, researchers have proposed a variety of methods. van de Ven et al. [25] taxonomise these methods into six computational mechanisms. *Replay* stores past examples in a buffer and interpolates them with new-task data, optimising the joint loss of the two. *Parameter regularisation* adds a penalty that anchors parameters identified as important for past tasks near their previous values, where importance is estimated from a curvature approximation of the loss [15]. *Functional regularisation* instead penalises changes to the network’s input–output mapping at a set of anchor points, typically through a distillation loss. *Optimisation-based* methods modify stochastic gra-

dient descent itself, seeking wider minima, adapting per-parameter learning rates, or projecting gradients to avoid interference. *Context-dependent processing* segregates tasks by routing computation per context, either by expanding the network or by gating parameters. *Template-based classification* sidesteps class-incremental decision boundaries by classifying against learned class prototypes, often in a frozen embedding space, rather than a trained softmax head. This thesis studies the replay family, and in particular its simplest member, Experience Replay.

2.2 Experience Replay

Experience Replay (ER) is the simplest replay baseline [4]. At each step it draws a replay mini-batch from the buffer, combines it with the current-task mini-batch, and takes a single optimiser step on the summed loss of the two. The buffer is populated by reservoir sampling [26], so every sample seen so far occupies a slot with equal probability regardless of arrival order. During T_0 the buffer is empty by construction, so ER reduces to plain SGD on the first task, and the stability gap is measurable only from T_1 onward.

This simplicity is why ER remains competitive. As Chaudhry et al. [4] show, jointly training with a small episodic memory often outperforms purpose-built methods, scaling gracefully to longer sequences without sacrificing plasticity like heavy regularisation does. Furthermore, because ER avoids explicitly manipulating gradients, it provides a clean setting to isolate the causes of the stability gap. We therefore adopt it as one of our two core baselines.

A finite buffer is an imperfect estimator of the true past-task distribution. Aljundi et al. [1] frame buffer selection as a constraint-reduction problem in which the stored samples approximate the feasibility region defined by the full past data; reservoir sampling is one (non-optimal) policy within that frame. The replay gradient computed on a small buffer therefore differs in direction and magnitude from the gradient on all past data, a discrepancy present at every step, independent of any task-boundary schedule.

2.3 Path-Finding Methods

A second class of replay-based and replay-adjacent methods constrains the SGD update at the task boundary using information about the past tasks. They differ in what information that is. GEM stores past-task gradients and projects the new-task gradient onto the nearest direction whose inner product with every stored gradient is non-negative, so the update does not increase the loss on any past task in the buffer [18]; this uses first-order (gradient) information only. A-GEM replaces the per-task constraints with a single averaged past-task gradient, reducing the projection to one cheap operation [3]. Both act only on the current step, not across the trajectory. EWC instead uses second-order information: it adds a quadratic penalty around the past optimum weighted by the diagonal of the Fisher information, slowing changes to parameters the old task constrained [15]. NCL also draws on curvature, preconditioning the update by the precision matrix of the past-task posterior to take a natural-gradient step along directions that leave the replay loss roughly unchanged [14]. In the framing of this thesis (Section 3.2), all four are *path-finding* methods: they accept the discontinuity at the task boundary and use stored gradient or curvature information to pick a safe trajectory across it. NCL is the second baseline we compare against.

These memory-based methods share a common update. Writing each step as $\theta \leftarrow \theta - \eta(\alpha_1 \nabla L_{\text{current}} + \alpha_2 \nabla L_{\text{replay}})$, a method is a rule for the relative weights α_1 and α_2 on the current-task and replay gradients [23]. Plain ER fixes $\alpha_1 = \alpha_2 = 1$. GEM and A-GEM hold $\alpha_1 = 1$ and set α_2 from the alignment between the current-task gradient and the stored past-task gradients, raising the replay weight only when the two conflict. MEGA [9] instead holds $\alpha_1 = 1$ and sets $\alpha_2 = L_{\text{replay}}/L_{\text{current}}$, shifting weight onto memory once the current task is already fit. In each case the weights are computed reactively, per step, from the interference the update is about to cause, which is the per-step character that makes them path-finding. The λ -curriculum of this thesis occupies the same update but moves the opposite weight: it holds $\alpha_2 \equiv 1$ and ramps $\alpha_1 = \lambda(t)$ from zero to one, replacing the reactive per-step rule with a predetermined schedule (the landscape-shaping alternative of Section 3.2); its adaptive variant rejoins

the reactive family as the mirror of MEGA, raising the current-task weight by the gradient-norm ratio $\|g_{\text{replay}}\|/\|g_{\text{new}}\|$ rather than the replay weight by a loss ratio (Section 6.2.2).

2.4 The Stability Gap

Lange et al. [17] identified the stability gap using continual evaluation, measuring past-task accuracy after every update rather than only at task boundaries. The gap is a temporary, sharp drop in past-task accuracy at the start of a new task, from which the model only partially recovers. This drop is most severe during the first few hundred steps. While larger replay buffers help reduce it, the gap survives reservoir sampling and moderate learning rates, and the minimum accuracy always falls below the final recovered baseline.

The literature traces this gap to two compound contributors. The first is a severe gradient imbalance at the task boundary: because the gradient is dominated by the new task, the optimiser’s initial steps prioritize plasticity and directly sacrifice old-task stability [17]. The second contributor is the underlying geometry of the descent path, which proves the gap is not just caused by imperfect memory. Hess et al. [12] show that the gap persists even when training on the exact joint loss with full access to all past data. Kao et al. [14] demonstrate this analytically: even with perfect joint gradients, steepest descent from an old-task minimum inherently forces the trajectory out of the old basin. Together, these results separate *what* is optimised from *how* it is optimised. A perfect objective function does not prevent the gap; the geometric trajectory (visualised in Figure 1.1) must be addressed directly.

The gap is broad rather than incidental: it appears in joint incremental learning of homogeneous tasks [13], in continual pre-training of large language models [8], and in relation to the classification head [27], and mitigations have been proposed, from temporal ensembles [24] to explicit gap-reduction procedures [10].

2.5 Blurry Task Boundaries

A parallel line of work relaxes the assumption that tasks arrive at sharp boundaries, replacing the instantaneous switch from T_0 to T_1 with a gradual change. The *blurry* setting, introduced with

Rainbow Memory [2], models this overlap explicitly. Classes from neighbouring tasks coexist in time, meaning samples of an upcoming class appear before the previous task ends. As a result, each task shades smoothly into the next rather than abruptly replacing it. In this setting, the replay buffer’s composition matters more than in disjoint streams because it must represent classes that are still arriving. Rainbow Memory addresses this by selecting stored samples for their *diversity* using an uncertainty-based scoring method. Combined with data augmentation, this approach outperforms standard disjoint-buffer baselines by a wide margin.

Online formulations push this boundary-free idea down to the level of individual samples. Koh et al. [16] study a class-incremental blurry stream under an *anytime-inference* constraint. Because the model may be queried at any point during training, it must remain accurate throughout the transition, rather than merely converging well at the end. Continuous evaluation surfaces the transient accuracy drops that conventional end-of-task metrics hide. This motivates both a metric that integrates accuracy over the entire stream and memory management designed to keep performance high at every step. In both settings, the core commonality is that the task transition spans many steps rather than occurring instantly.

This gradual transition is what connects these settings to our intervention. In a blurry stream, a mini-batch shifts progressively from almost entirely T_0 data to almost entirely T_1 data over the transition window. Consequently, the gradient the optimiser follows is a moving mixture that shifts gradually from the old task to the new. Our λ -curriculum produces this exact same moving mixture, but through the *loss weight* rather than the *data*. It scales the new-task loss term up from zero while holding the replay term fixed. Up to an overall scale, both methods descend the same objective. Both keep the past-task gradient present from the very first step while introducing the new-task gradient gradually.

2.6 Linear Mode Connectivity

The introduction claimed that a low-loss corridor connects the single-task solution to the joint optimum. Work on *mode connectivity* supports this.

Garipov et al. [7] and Draxler et al. [5] showed that independently trained minima of a deep network, though separated by a barrier along the straight line between them, are joined by curved paths of near-constant low loss. Frankle et al. [6] sharpened this to *linear* mode connectivity: networks that share a short period of early training can be joined by a straight low-loss path. For continual learning specifically, Mirzadeh et al. [21] found that the multitask solution and the sequentially trained solution often lie in the same basin, linearly connected, when the training regime is stable. This matters for the stability gap in two ways. It means the dip is not forced by the geometry, because a low-loss route from θ_A^* to θ_{joint}^* exists; and it sharpens what an intervention has to do, namely keep the optimiser on that route rather than let it bow away, as the trajectory argument predicts it otherwise will.

3 Motivating a Landscape-Shaping Approach

This thesis defends one claim: in its dominant components, the stability gap is the downstream symptom of the discontinuity at the task boundary (Section 1). We make the claim precise in two steps. First we decompose the gap into three contributors (Section 3.1). We then contrast the two ways one can intervene on the discontinuity (Section 3.2), which is what motivates the landscape-shaping approach the rest of the thesis develops.

3.1 Decomposing the Stability Gap

We unify the causes scattered across the literature — gradient imbalance, the finite-buffer estimator, and the trajectory effect (Sections 2.2 and 2.4) — into a single decomposition of three forces. We isolate and measure each empirically in Section 6.

Magnitude asymmetry (G_{mag}). Consider the first step of vanilla Experience Replay (ER) at the start of a new task, T_1 , taken from the previous task’s optimum θ_0^* . Because θ_0^* is a stationary point of the replay loss, the true replay gradient there is near zero, so the update is driven almost entirely by the new-task gradient. This large and unopposed new-task gradient carries the iterate out of the replay basin. The replay loss rises the further the iter-

ate moves from θ_0^* , until it grows strong enough to pull the iterate back and the old task is relearned. Because the imbalance is fixed by the relative gradient magnitudes at this first step, it is the dominant contributor to the gap, and corresponds to the gradient-imbalance effect observed by Lange et al. [17].

Estimator variance (G_{est}). In any individual training run, the replay buffer provides only a finite-sample estimate of the past data (Section 2.2) [1]. This per-step sampling noise creates a baseline loss degradation that persists regardless of the learning schedule, as it is independent of how the loss terms are composed.

Trajectory effect (G_{traj}). Even in an idealized scenario where magnitude asymmetry is corrected and buffer noise is eliminated, a geometric penalty remains: an instantaneous switch in objectives forces the iterate off the optimal path of the joint loss (Section 2.4). Along this new trajectory, the replay loss is strictly higher than it would be at the joint-loss optimum θ_{joint}^* .

Crucially, these three components dominate at different timescales: G_{mag} drives the massive disruption at step one, G_{traj} dictates the curve over the first equilibration window, and G_{est} provides constant noise across all steps.

3.2 Path-Finding versus Landscape-Shaping

There are two ways to intervene, distinguished by what they do with the discontinuity.

Path-finding: accepts the discontinuity. The objective still jumps at the boundary, from the replay loss alone to the sum of the replay and current-task losses, and the intervention instead reshapes *how* the optimiser moves across that jump. Rather than taking a plain gradient step, it reweights each update using information about the past tasks, steering the optimiser away from directions that would sharply raise the replay loss. This past-task information takes different forms: the curvature estimate of EWC [15], the posterior precision of NCL [14], or the gradient projection of GEM [18]. The path-finding methods of Section 2.3 all sit here.

Landscape-shaping: removes the discontinuity. It modifies the objective near the boundary so the surface the optimiser descends is continuous through the transition, leaving the geometry of each step to SGD’s natural step-size adaptation. The λ -curriculum is the canonical example: instead of switching the objective from the replay loss to the joint loss in a single step, it ramps the weight on the current-task loss up from zero to one over a continuous schedule, while holding the replay loss fixed. Two consequences distinguish it from path-finding. A preconditioner rescales the gradient and so breaks SGD’s step-size adaptation, whereas the curriculum only reweights the loss and leaves the step size untouched; and where a preconditioned update must build past-task information into every step, the curriculum adds nothing more than a single changing weight. Section 4 shows that this continuity is enough to remove the trajectory contributor G_{traj} , and that it simultaneously tempers G_{mag} by introducing the new-task gradient gradually rather than all at once.

4 The Envelope-Theorem Bound: Why a Linear λ -Ramp Works

When a model switches instantly from an old task to a new one (λ jumps from 0 to 1), the sudden change causes the stability gap with a temporary spike in forgetting.

This section outlines why replacing that sudden jump with a slow, continuous transition (a linear ramp over N steps) mathematically eliminates this spike. The full, step-by-step mathematical derivation of this proof is provided in Appendix D.

The core argument is built on two pillars:

4.1 Part A: The perfect path never overshoots

Imagine an “optimum path” ($\Theta^*(\lambda)$) that perfectly tracks the exact bottom of the combined loss valley as the curriculum transitions from pure replay ($\lambda = 0$) to the joint task ($\lambda = 1$). Figure 4.1 shows this path directly: as λ grows, the minimiser of the curriculum loss $L_\lambda = A + \lambda B$ migrates smoothly from the old-task optimum A to the joint optimum

$A+B$ along the linear path, so a model that follows it rides a moving valley floor rather than jumping between the two.

If the model were capable of perfectly walking this ideal path, the replay loss would never experience a sudden spike. As proven in the Appendix D using the envelope theorem, the rate of change of the replay loss along this path is mathematically guaranteed to be non-negative:

$$\frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) \geq 0 \quad \text{for all } \lambda \in [0, 1]. \quad (4.1)$$

Because the replay loss can only strictly rise (or stay flat) as it accommodates the new task, it can never decrease. Therefore, it is physically impossible for the loss to spike up and come back down. It simply rises monotonically until it reaches its permanent, unavoidable baseline at the joint optimum (θ_{joint}^*).

This argument needs no global convexity. It requires only that the tracked path stays a family of *local* minima — that the combined Hessian remains positive-definite along the branch the iterate follows — which holds automatically near $\lambda = 0$ and persists until the new task’s negative curvature overpowers the old task’s, the point at which the bound is expected to weaken (Appendix D).

4.2 Part B: A slow ramp keeps the model on the path

Part A describes a mathematically perfect path, but real SGD iterates ($\theta(t)$) are clumsy and take discrete steps. However, by analyzing the gradient flow, we can prove that the actual model trajectory acts as though it is tethered to the perfect path.

The distance the model lags behind the perfect path is directly proportional to how fast the valley floor is moving. If the transition is stretched over a sufficiently long ramp (large N), the maximum distance the model wanders off the ideal path is tightly bounded:

$$\|\theta(t) - \Theta^*(\lambda(t))\| = O\left(\frac{1}{N}\right). \quad (4.2)$$

4.3 The Combined Bound

By combining the “no-overshoot” rule of the perfect path (Part A) with the tracking error of the actual

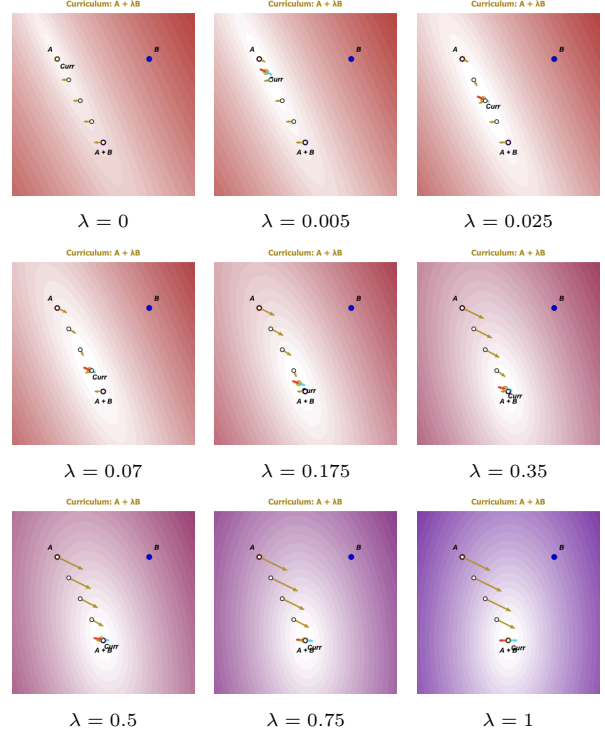


Figure 4.1: Evolution of the curriculum loss $L_\lambda = A + \lambda B$ over a shared two-dimensional parameter space. The weight on the new task ramps from $\lambda = 0$ (pure replay) to $\lambda = 1$ (the joint loss); the value of λ is printed beneath each panel. A and B mark the two single-task minima and $A+B$ the joint minimum. As λ increases the contours morph continuously from the old-task basin around A to the joint basin around $A+B$, and the current minimiser ($Curr$) slides along the linear path from A toward $A+B$. Which is the optimum path $\Theta^*(\lambda)$ of Section 4.1. The opposing red and blue arrows at the current minimiser are the replay- and current-task gradient contributions, which cancel there by the first-order condition $\nabla L_{\text{replay}} + \lambda \nabla L_{\text{current}} = 0$. Because the minimiser never jumps, it only drifts as λ grows, a model that tracks it rides the moving valley floor instead of being teleported across the landscape.

network (Part B), we arrive at the final bound for the highest replay loss the model will experience during the transition:

$$\max_t L_{\text{replay}}(\theta(t)) \leq L_{\text{replay}}(\theta_{\text{joint}}^*) + O\left(\frac{1}{N}\right). \quad (4.3)$$

The first term is the genuine plasticity–stability trade-off: the permanent capacity cost of learning both tasks. The second term is the transient stability gap.

Read in the other direction, the same two parts explain why a *small* N — in the limit the instantaneous jump — produces a large gap, through two distinct mechanisms that map onto the two empirical readouts. The one-sided first step, driven by the new-task gradient while the replay gradient is still zero, gives an instantaneous curvature-cost spike that scales as $1/N^2$ and registers as `gap_depth`; the trajectory lag behind the moving optimum is sustained across the whole ramp, scales as $1/N$, and registers as the slowly-closing `gap_area`. At the jump both are $O(1)$ and the gap is maximal. The full decomposition is in Appendix D.

Because the transient gap shrinks proportionally with $1/N$, a sufficiently slow continuous curriculum physically drives the trajectory overshoot to zero. The bound formally concerns only the trajectory contributor G_{traj} , but the same gradual introduction also tempers the magnitude asymmetry G_{mag} : ramping the current-task loss up from zero lets its gradient enter in small increments rather than dominating the first step, so the iterate is never thrown as far out of the replay basin. Only the estimator variance G_{est} , a property of the finite buffer rather than the schedule, is left untouched. The model’s step-size adaptation is preserved throughout, cleanly resolving the gap without the need for complex path-finding interventions.

5 Research Questions

The decomposition of Section 3.1 and the bound of Section 4 make a set of falsifiable predictions about how the λ -curriculum and its refinements should behave. We test them on rotated MNIST with a small multilayer perceptron and then ask whether the findings survive a deeper backbone and a stronger task shift. The experiments are organised around three primary questions, the second of

which carries four sub-questions probing the individual ingredients of the curriculum.

RQ1 — The cause. Does the stability gap decompose into the magnitude, estimator, and trajectory contributions of Section 3.1, and is the discontinuity their dominant source?

RQ2 — The fix. Does removing the discontinuity with the λ -curriculum reduce the stability gap without final-accuracy cost? Four sub-questions probe how:

RQ2a (linear ramp). Does the linear curriculum reduce gap depth monotonically as the ramp length N grows?

RQ2b (adaptive schedule). Does an adaptive schedule achieve the same reduction while removing the explicit ramp-length knob N ?

RQ2c ($\lambda_{\min}$ floor). Does a small floor $\lambda_{\min} > 0$ recover early-task learning speed without reintroducing the depth the curriculum was designed to remove?

RQ2d (momentum). Does momentum suppress the residual depth fluctuation on top of the curriculum, while (as a control) doing nothing on its own, since the first step at the boundary is deterministic?

RQ3 — Generalisation. Do these findings carry over to a deeper backbone (ResNet-18) and a stronger task shift (CIFAR-10 domain corruption)?

Section 6 describes the testbed, the metrics used to measure gap depth and final accuracy, and the full set of conditions that isolate each contributor and test each prediction.

6 Methods

This section describes the empirical testbed (Section 6.1.1), the metrics that summarise each run (Section 6.1.3), and the full set of conditions (Sections 6.2–6.2.4) that test the predictions of Sections 3 and 4. Each experimental block maps to one of the research questions of Section 5 and to a falsifiable prediction derived earlier.

6.1 Experimental Setup

6.1.1 Testbed

The experiments run on two testbeds of deliberately different scale. A small, fast testbed — rotated MNIST on a two-layer MLP — carries the dense mechanistic sweep, where the gap is probed step by step across many conditions and seeds. A larger testbed — a domain-shift CIFAR-10 on a CIFAR-adapted ResNet-18 — tests whether the findings survive a deeper backbone and a stronger task shift, the regime where the smoothness assumption behind the bound of Section 4 weakens. Both testbeds share one design choice: the label space and task structure are held fixed and only the input distribution shifts, and the headline sequences use the minimal two tasks, so there is exactly one transition and exactly one instance of the stability gap, with no interference between overlapping gaps.

Datasets. The small benchmark is rotated MNIST (rot-MNIST). Following Lange et al. [17], rotations are measured from a -90° baseline: the first task T_0 presents the digits at -90° , and the second task T_1 applies a further 90° rotation to bring them to 0° . Each task uses the standard 60,000 training and 10,000 test images. The 90° rotation makes the two input distributions well separated while keeping the tasks structurally identical, with the same label space and the same underlying digit identities. A five-task variant (rotations $[-90, -60, -30, 0, 30]^\circ$) is used for the long-sequence block of Section 6.2.3.

The large benchmark is a domain-shift CIFAR-10: T_0 uses clean CIFAR-10 and T_1 uses the Gaussian-noise corruption at severity 3. The ten classes are shared, so as in rot-MNIST only the input distribution shifts and two tasks produce a single transition. A three-task variant ([none, Gaussian noise, shot noise]) and two alternative corruptions are used in the generalisation block of Section 6.2.4.

Models. Each benchmark has its own backbone. The rot-MNIST sweep uses a two-layer multi-layer perceptron (MLP) with ReLU activations (Table 6.1); the CIFAR-10 block uses a CIFAR-adapted ResNet-18 (Table 6.2).

Component	Value
Input	784 (flattened 28 × 28 greyscale)
Hidden layers	2 × 400 (fully connected, ReLU)
Output	10-class logits
Regularisation	none
Total parameters	478,410

Table 6.1: MLP backbone used for the rot-MNIST sweep.

The MLP’s loss landscape is simple and it trains in seconds, which matters because the sweep spans many conditions and seeds. It is also the setting where the assumption behind the bound of Section 4 is most plausible, that near the optimum the loss is close to a smooth, locally convex bowl.

Component	Value
Input	3 × 32 × 32 RGB
Stem	3 × 3 conv, stride 1, no max-pool (CIFAR adaptation)
Stages	4 residual stages, BasicBlocks [2, 2, 2, 2], widths [64, 128, 256, 512]
Normalisation	BatchNorm (default); GroupNorm (min(32, C) groups) variant crossed in Section 6.2.4
Head	global average pool + linear, 10-class logits
Total parameters	≈11.2M

Table 6.2: CIFAR-adapted ResNet-18 backbone used for the CIFAR-10 block (Section 6.2.4). The 3 × 3 stride-1 stem and removed max-pool keep the feature map at 32 × 32; all other details follow the canonical ResNet-18.

ResNet-18 is the deeper, harder-to-analyse counterpart: it is used to see how far the findings carry once the locally convex picture is no longer guaranteed.

Training protocol. Table 6.3 lists the training hyperparameters for both benchmarks. rot-MNIST runs in the online regime of one epoch per task, standard for stability-gap analysis: it gives a single clean pass through the transition, where the gap is most pronounced. ResNet-18 on CIFAR-10 needs more than one pass to converge before the

transition, so the CIFAR block uses ten epochs per task throughout; everything else (optimiser, learning rate, batch size) is held common across the two benchmarks.

Hyperparameter	rot-MNIST (MLP)	CIFAR-10 (ResNet-18)
Epochs per task	1 (online)	10
Batch size	256	256
Optimiser	SGD	SGD
Learning rate	0.1	0.1
Momentum	0.0 and 0.9 (both legs)	0.9 only
Weight decay	0.0	0.0
Replay buffer	1,000 (reservoir); 60,000 (full-data conditions)	5,000 (reservoir)
Seeds	5 per condition	5 (headline); 3 (generalisation)

Table 6.3: Training protocol for both benchmarks. The 60,000-sample full-data buffer is used only for the full-data decomposition conditions of Section 6.2.1. The rot-MNIST momentum policy is detailed in Section 6.2.

6.1.2 Evaluation protocol

Throughout training, every task’s test accuracy is evaluated every 10 steps. At each task transition the cadence increases to every step, so the dip is recorded at full resolution where it occurs. The two benchmarks differ only in how long this per-step cadence is held. On rot-MNIST it runs for the entire new task: one online epoch is short (≈ 234 steps), so the whole post-transition trajectory is captured per step. On CIFAR-10, where a task spans ten epochs, the per-step cadence is held for the first 50 steps after the transition, enough to cover the gap, and then drops back to every 10 steps for the rest of the task.

A stability-gap tracker snapshots the pre-task accuracy on each past task before the transition and records the resulting (step, accuracy) series, from which it computes the scalar summaries of Section 6.1.3. A gradient tracker records, at the same per-step cadence, the directional and magnitude fidelity of the replay-buffer gradient against the deterministic empirical past-task gradient g_{true}

(Section 6.1.3); it is enabled for the decomposition conditions only.

6.1.3 Metrics

All metrics are recorded automatically and aggregated later. Unless noted otherwise, each is reported as the seed mean $\pm$ sample standard deviation over a condition’s seeds (five for the rot-MNIST and CIFAR headline blocks, three for the CIFAR generalisation block); **gap_area** is reported as a seed mean only, and **recovery_steps** only when a recovery occurs within the record. This section explains the most important ones; the complete definitions of every logged metric are collected in Appendix B.

Stability-gap metrics. These are computed from the dense per-step series. Writing a_j^{pre} for the pre-task accuracy on past task j and $a_j(t)$ for its accuracy at step t of the new task, the headline measure is

$$\text{gap_depth} = \max_j \left(a_j^{\text{pre}} - \min_t a_j(t) \right),$$

the worst single point of instability across the record window. It is the primary outcome in every contrast: each prediction of Sections 3 and 4 is stated as a claim about how **gap_depth** moves between conditions. A companion scalar, **gap_area**, integrates the past-task accuracy shortfall over the new-task record, but measured against the level each task *recovers to* — the trailing-window mean of its own curve, floored at the pre-switch level a_j^{pre} — rather than against a_j^{pre} itself. This makes it a measure of the transient cost of the gap: the dip below the recovered level that later closes, whether that dip is a sharp notch or a gradual bend. A task that drifts to a lower plateau without such a dip (permanent forgetting) contributes ≈ 0 , as does one still improving at the switch; only a genuine dip-and-recover registers. Permanent retention loss is read off ACC/FORG instead, so depth, area, and the accuracy metrics separate how deep the gap goes, how much it costs before recovering, and the permanent level. **recovery_steps** reports how many steps after the transition every past task first returns to within 90% of its pre-task baseline, and is undefined when no sub-threshold dip occurs. All are reported alongside depth but are secondary. Their

precise definitions, and the fixed-window `max_drop` kept only for back-comparability, are in Table B.1.

Continual-learning metrics. To place the gap in the standard continual-learning frame, we also report the continual-evaluation metrics of Lange et al. [17], computed from the task-boundary accuracy matrix $R[i, j] = A(E_j, f_{T_i})$ that is logged for every run (so any downstream metric can be reconstructed). The two that carry the argument are average final accuracy (ACC), which checks that a curriculum buys lower depth without an accuracy cost, and the worst-case measures min-ACC and WC-ACC, which report retention at its lowest point rather than only at the end. FORG and the windowed worst-case forgetting/plasticity measures WF_w/WP_w are reported for completeness; all definitions are in Table B.2.

Gradient-fidelity diagnostics. For the decomposition block the gradient tracker compares the replay-buffer gradient g_{replay} against the deterministic empirical past-task gradient g_{true} (a separate full-data forward/backward pass, no subsampling). Two diagnostics measure how faithfully a finite buffer approximates the past data [1]: the directional fidelity $\cos(g_{\text{replay}}, g_{\text{true}})$ and the magnitude fidelity $\|g_{\text{replay}}\|/\|g_{\text{true}}\|$. These supply the direct per-step evidence for the estimator-bias contributor of Section 6.2.1. A third, $\|g_{\text{new}}\|/\|g_{\text{replay}}\|$, is the update-side magnitude asymmetry; its inverse $\|g_{\text{replay}}\|/\|g_{\text{new}}\|$ is the signal the adaptive curriculum of Section 6.2.2 uses to set $\lambda(t)$. Logged names and exact definitions are in Table B.3.

Each diagnostic step adds one forward and backward pass over every past-task training sample (60,000 for rot-MNIST T_0), which is affordable on the MLP testbed but would dominate runtime on larger backbones, so the toggle stays off for the curriculum and CIFAR blocks, whose claims rest on `gap_depth` and ACC rather than per-step gradient geometry.

Statistical comparison. Across conditions we compare metrics with the paired Wilcoxon signed-rank test on the seed-paired difference. Seeds are paired by index across conditions, so the per-seed deltas $\Delta_s = m(C_a, s) - m(C_b, s)$ factor out the seed-induced variance that would otherwise dominate

the five-seed sample. We prefer Wilcoxon over the paired t -test because `gap_depth` and accuracy are bounded in $[0, 1]$ and typically right-skewed near zero, so the t -test’s normality assumption is suspect; the Wilcoxon test exchanges that assumption for the milder one that Δ_s is distributionally symmetric under the null. For equivalence-style claims, such as “the adaptive curriculum matches the best linear curriculum”, we report the seed-paired difference with its bootstrap 95% confidence interval rather than a null-hypothesis test.

6.2 Experiments

Each block tests one or more predictions of Sections 3 and 4 and answers one research question (Table 6.4). All conditions use five seeds and the training protocol of Section 6.1.1 unless noted otherwise.

Momentum. Every rot-MNIST condition, the whole decomposition G-series and the whole curriculum C-series, is run twice, once at momentum 0.0 (plain SGD) and once at momentum 0.9; the momentum-on runs carry an M subscript (e.g. $G1_M$) and are reported beside their $\mu=0$ counterparts. The long-sequence and CIFAR-10 blocks ship a single momentum setting (0.9, the better-performing leg from rot-MNIST), since the two-task sweep already characterises the momentum trade-off at a transition.

6.2.1 Three-Contributor Decomposition

Aim. Isolate the three contributors of Section 3.1 — magnitude asymmetry (G_{mag}), estimator variance (G_{est}), and the trajectory effect (G_{traj}) — and quantify each one’s share of the total gap. This is the diagnostic question of RQ1.

Design. Two of the three contributors are predicted to be downstream symptoms of the discontinuity at the task boundary; the third, estimator variance, is a property of finite replay buffers and persists at any schedule. Removing each contributor by construction and observing the residual gap depth identifies which contributors drive the dip. The path $G1 \rightarrow G2 \rightarrow G3 \rightarrow G4$ removes one source at a time (Table 6.5).

Block	Section	RQ	Falsifiable prediction	Conditions
Three-contributor decomposition	6.2.1	RQ1	Magnitude, estimator, and trajectory together account for the total gap; the trajectory residual is small but non-zero	G1–G4, plus NCL reference
Linear λ -curriculum	6.2.2	RQ2a	<code>gap_depth</code> falls monotonically with N ; ACC matches vanilla ER	C1–C3 ($N \in \{50, 100, 200\}$)
Adaptive λ -curriculum	6.2.2	RQ2b	<code>gap_depth</code> falls relative to vanilla ER with no explicit knob N	C4 (adaptive from $\ g_{\text{replay}}\ /\ g_{\text{new}}\ $)
$\lambda_{\min}$ refinement	6.2.2	RQ2c	A floor $\lambda_{\min} > 0$ recovers early velocity without inflating depth to vanilla-ER levels	C5–C7 ($\lambda_{\min} \in \{0.05, 0.10, 0.20\}$)
Momentum (cross-cut)	6.2.2	RQ2d	Momentum tightens depth on top of the curriculum, but momentum alone does not reduce depth	every rot-MNIST condition at $\mu \in \{0.0, 0.9\}$
Long-sequence rot-MNIST	6.2.3	all	The curriculum holds across several transitions, not just one	L1–L4 (vanilla ER, NCL, adaptive $\lambda_{\min} \in \{0, 0.10\}$); 5 tasks
CIFAR-10 generalisation	6.2.4	all	Headline contrasts carry to a stronger task shift and a deeper backbone	D1–D6 (vanilla ER, NCL, adaptive $\lambda_{\min}=0.20$, each at BatchNorm and GroupNorm); plus a novel-corruption block

Table 6.4: Overview of the experimental blocks, the research question each answers, and the prediction each tests.

Condition	Replay buffer	Gradient handling	Removed by construction	Residual gap
G1 — Vanilla ER	1k reservoir	unbalanced	nothing	magnitude + estimator + trajectory
G2 — Full-data ER	60k (all past)	unbalanced	estimator noise (exact gradient)	magnitude + trajectory
G3 — Balanced ER	1k reservoir	unit-norm balanced	magnitude asymmetry	estimator + trajectory (+ balancing cost)
G4 — Full-data + balanced	60k	unit-norm balanced	magnitude and estimator	trajectory
NCL (reference)	none	K-FAC precision preconditioning	—	path-finding reference [14]

Table 6.5: Decomposition conditions. Each row removes one source of the gap relative to G1, so the residual gap shrinks down the path $G1 \rightarrow G3 \rightarrow G4$.

Gradient balancing. The balanced conditions G3 and G4 compute the replay and new-task gradients in two separate backward passes, normalise each to unit norm, and combine them 50/50, so the two terms enter the update with equal magnitude regardless of their raw sizes. This targets the first-step asymmetry of Section 3.1: at θ_0^* the replay gradient is near zero, so an unbalanced step is driven almost entirely by g_{new} , whereas after normalisation the replay direction carries equal weight from the first step. Balancing removes the magnitude asymmetry by construction, so G_{mag} is exactly the gap-depth difference between an unbalanced step (G1) and a balanced one (G3).

Full-data conditions. The full-data conditions G2 and G4 hold the entire past-task training set in the buffer and, at every step, compute the replay gradient on every stored sample exactly once. The resulting replay gradient is the deterministic empirical past-task gradient at the current parameters — no sampling, no bootstrap variance. Vanilla ER (G1) draws 256 samples with replacement, so its replay gradient is a stochastic estimator of this same quantity. The estimator contributor G_{est} is therefore exactly the gap-depth difference between an estimator-driven step and an exact-gradient step, and G_{traj} in G4 is the residual once both the estimator and the magnitude asymmetry are removed.

Decomposition shares. Differences in `gap_depth` along the path identify each contributor. The magnitude share (G_{mag}) is the drop from G1 to G3, removed by balancing alone; the estimator share (G_{est}) is the further drop from G3 to G4, the additional reduction from full data on top of balancing; and the trajectory share (G_{traj}) is the residual gap at G4 once both are removed, the trajectory contribution of Section 4. The G2 condition is a sanity check: `gap_depth(G1) - gap_depth(G2)` should recover G_{est} along an orthogonal path through the design.

NCL reference. NCL is included as a path-finding baseline (Section 3.2): it accepts the discontinuity but preconditions each SGD step by the past-task precision matrix.

Prediction. Gap depth reduces along the path $G1 \rightarrow G3 \rightarrow G4$. The trajectory residual at G4 is small but non-zero (the prediction of Kao et al. [14], Section 2.4), and is the contributor the λ -curriculum should additionally suppress.

6.2.2 The λ -Curriculum Sweep

Aim. Test the envelope-theorem prediction of Section 4 — that a continuous λ -schedule eliminates the trajectory contribution to the gap, with a transient that shrinks as $O(1/N)$ (Equation (4.3)) — the $\lambda_{\min}$ trade-off, and the momentum cross-cut of RQ2d. Answers RQ2a–RQ2d.

Design. All seven curriculum conditions (labelled C1–C7) are applied on top of standard ER (1k reservoir, no balancing, no NCL) so the curriculum’s effect is isolated (Table 6.6). As stated in Section 6.2, each is run at both momentum 0.0 and 0.9.

The adaptive schedule (C4) is self-paced. At θ_0^* the replay gradient is near zero by stationarity, so the ratio $r := \|g_{\text{replay}}\|/\|g_{\text{new}}\|$ is near zero and λ starts near 0. As the iterate leaves θ_0^* , $\|g_{\text{replay}}\|$ grows while $\|g_{\text{new}}\|$ shrinks as the model fits T_1 , so r climbs and $\lambda \rightarrow 1$ with no explicit ramp length; the schedule ignores N entirely. The floor $\lambda_{\min}$ (C5–C7) matters mainly for this adaptive variant: it stops the schedule from sitting at $\lambda \approx 0$ in the first steps, when the EMA estimate of r is still close to zero.

Prediction. `gap_depth` should fall monotonically as N grows from 50 to 200 (the $O(1/N)$ bound of Equation (4.3)), while ACC stays close to vanilla ER, because the curriculum preserves SGD’s natural step-size adaptation rather than rescaling the step the way gradient balancing does (RQ2a). The adaptive schedule should reach a comparable depth reduction without fixing N (RQ2b). A small floor $\lambda_{\min} \in [0.05, 0.15]$ should restore early T_1 velocity — the adaptive schedule otherwise sits near $\lambda \approx 0$ in the first steps and barely trains the new task — giving faster early learning and higher final accuracy, at the cost of only a slight increase in gap depth (well short of vanilla-ER levels), for a better (depth, ACC) Pareto position than $\lambda_{\min} = 0$ (RQ2c).

The contrast between balanced ER (G3) and linear $N=200$ (C3) isolates the difference between

Label	Condition	$\lambda(t)$ schedule	Hyperparameter	Tests
—	Vanilla ER (base-line)	$\lambda \equiv 1$ from step 1 (discontinuous)	—	reference
C1	Linear $N=50$	$\text{clip}(t/50, 0, 1)$	$N = 50$	RQ2a
C2	Linear $N=100$	$\text{clip}(t/100, 0, 1)$	$N = 100$	RQ2a
C3	Linear $N=200$	$\text{clip}(t/200, 0, 1)$	$N = 200$	RQ2a
C4	Adaptive	$\text{clip}(\text{EMA}_\alpha(\ g_{\text{replay}}\ /\ g_{\text{new}}\), 0, 1)$	$\alpha = 0.05$	RQ2b
C5	Adaptive + $\lambda_{\min}=0.05$	$\max(\lambda_{\min}, \text{clip}(\text{EMA}_\alpha(\ g_{\text{replay}}\ /\ g_{\text{new}}\), 0, 1))$	$\alpha = 0.05, \lambda_{\min}=0.05$	RQ2c
C6	Adaptive + $\lambda_{\min}=0.10$	as C5	$\lambda_{\min}=0.10$	RQ2c
C7	Adaptive + $\lambda_{\min}=0.20$	as C5	$\lambda_{\min}=0.20$	RQ2c

Table 6.6: λ -curriculum conditions (C-series), all applied on top of standard ER. EMA_α is an exponential moving average with smoothing α . Each condition is run at momentum 0.0 and 0.9.

acting on the magnitude symptom alone and acting on the schedule discontinuity: both should suppress depth, but only the curriculum should preserve ACC.

Momentum (RQ2d). Because every condition above (and the decomposition G-series) is run at both momentum 0.0 and 0.9, the same sweep tests the two momentum predictions of RQ2d. On top of the curriculum, momentum should reduce the seed-to-seed standard deviation in **gap_depth** and the per-step oscillation amplitude on the T_0 accuracy curve, without changing the depth mean by much — the residual it cleans up is the stochastic oscillation around the optimum path. Without the curriculum, momentum should *not* reduce **gap_depth**: depth for vanilla ER (G1) at momentum 0.9 should match depth at momentum 0.0, because the one-sided first step at the boundary is deterministic and its direction is unchanged. The contrast C3 vs. C3_M measures the residual momentum removes; the contrast G1 vs. G1_M tests the falsification, and a null there is the falsifiable prediction.

6.2.3 Long-Sequence rot-MNIST

Aim. The two-task design isolates a single transition, which keeps the decomposition clean but leaves two questions open: whether the curriculum still suppresses the gap across several consecutive boundaries, and whether NCL recovers the advantage that Kao et al. [14] report at the higher task counts where path-finding methods are strongest.

Design. A five-task rot-MNIST sequence with rotations $[-90, -60, -30, 0, 30]^\circ$, giving four consecutive transitions over a uniform 30° step. The backbone, online regime, and dense per-step evaluation are those of the two-task block (Section 6.1.1). Momentum is fixed at 0.9: the C-series and the G1/G1_M contrast already characterise the momentum trade at a single transition, so running both legs here would add little. Four conditions (Table 6.7) — vanilla ER, tuned NCL, and the adaptive curriculum at $\lambda_{\min} \in \{0, 0.10\}$ — each at five seeds. Gap-depth and accuracy are averaged over the four transitions.

Label	Method	$\lambda_{\min}$
L1	Vanilla ER	—
L2	NCL (tuned)	—
L3	ER + adaptive curriculum	0.00
L4	ER + adaptive curriculum	0.10

Table 6.7: Five-task rot-MNIST conditions (rotations $[-90, -60, -30, 0, 30]^\circ$, four transitions, momentum 0.9).

Prediction. The adaptive curriculum should keep a reduced gap depth across all four transitions at no accuracy cost relative to vanilla ER. NCL may reach a lower depth at this higher task count, consistent with Kao et al. [14], but is expected to carry the accuracy cost it shows at a single transition.

6.2.4 Generalisation: CIFAR-10

Aim. Confirm the direction of the rot-MNIST findings under a stronger task shift and a deeper, harder-to-analyse architecture — the regime in which the assumptions behind the envelope-theorem bound weaken (Section 4). The headline dataset is the two-task Gaussian-noise domain shift of Section 6.1.1.

Training protocol. The CIFAR-adapted ResNet-18 of Table 6.2, ten epochs per task — at this depth ResNet-18 needs more than one pass to converge before the transition — batch size 256, SGD with learning rate 0.1, and a replay buffer of 5,000 samples. Momentum is fixed at 0.9 (the better-performing leg from rot-MNIST); the two-leg momentum sweep of Section 6.2 is not repeated here. Evaluation follows the scheme of Section 6.1.2: per-step for the first 50 steps after the transition, every 10 steps otherwise.

Headline conditions. The headline block is six conditions: three methods — vanilla ER, NCL, and the adaptive curriculum at $\lambda_{\min} = 0.20$ — each run under both BatchNorm and GroupNorm (Table 6.8). A linear curriculum is not run on CIFAR; $\lambda_{\min} = 0.20$ is the single carried-over curriculum setting, the adaptive variant that gave the best final accuracy in the rot-MNIST $\lambda_{\min}$ sweep. NCL hyperparameters are pinned to the values that won the rot-MNIST tuning sweep (Appendix A). The two normalisations are crossed in deliberately: they govern the post-switch recovery dynamics. Batch-Norm running statistics are disrupted by the input shift and re-adapt over the first steps of the new task, whereas GroupNorm carries no cross-batch statistics, so the pair separates the normalisation-driven part of the dip from the optimisation-driven part the curriculum targets.

Label	Method	Norm	Tests
D1	Vanilla ER	BatchNorm	reference
D2	NCL (tuned)	BatchNorm	reference
D3	Adaptive $\lambda_{\min}=0.20$	BatchNorm	carry-over
D4	Vanilla ER	GroupNorm	reference
D5	NCL (tuned)	GroupNorm	reference
D6	Adaptive $\lambda_{\min}=0.20$	GroupNorm	carry-over

Table 6.8: CIFAR-10 headline conditions: three methods crossed with two normalisations, buffer 5,000, momentum 0.9, five seeds. The shipped curriculum is the adaptive schedule at $\lambda_{\min} = 0.20$.

Generalisation block. A second CIFAR block tests whether the headline finding depends on the particular corruption or on having only a single transition. It pairs the shipped curriculum (adaptive $\lambda_{\min} = 0.20$, buffer 5,000) against a vanilla-ER control on three further settings: a two-task shot-noise shift, a two-task contrast shift, and a three-task sequence [none, Gaussian noise, shot noise] (Table 6.9). Each shift is run at both normalisations over three seeds. The paired vanilla control on every shift keeps the comparison interpretable: the question is whether the curriculum still beats vanilla on data it was not tuned on.

Shift	Tasks	Conditions
Shot noise	2	ship vs. vanilla
Contrast	2	ship vs. vanilla
Gaussian noise + shot noise	3	ship vs. vanilla

Table 6.9: CIFAR-10 generalisation block. Each shift is run with the ship config (adaptive $\lambda_{\min}=0.20$) and a paired vanilla-ER control, at both BatchNorm and GroupNorm, three seeds.

Prediction. Qualitative carry-over of the rot-MNIST result: the adaptive curriculum should reduce gap depth relative to vanilla ER and at least match NCL on depth while preserving ACC, under both normalisations, and should keep beating its vanilla control on the novel corruptions of the generalisation block. The quantitative $O(1/N)$ scaling of the bound is not expected to hold tightly at this depth; deviations are reported and discussed along-

side the results.

7 Results

This section reports the outcome of the experimental blocks of Section 6, in the order of the research questions of Section 5: the three-contributor decomposition (Section 7.1, RQ1), the λ -curriculum sweep on rot-MNIST (Section 7.2, RQ2a, RQ2b, RQ2c), the momentum cross (Section 7.3, RQ2d), the CIFAR-10 generalisation block (Section 7.4, RQ3), and the long-sequence rot-MNIST check (Section 7.5).

How to read the numbers. The headline metric is `gap_depth`, the worst single-step drop in past-task accuracy at the transition (Section 6.1.3), in percentage points (pp): a depth of 19.5 means the worst evaluation point sat 19.5 points below the pre-task baseline. Depth captures only how far the dip falls. It says nothing about how much accuracy the gap costs before the curve reconverges — which `gap_area` measures: the dip integrated against the level each task recovers to, so a permanent drop to a lower plateau does not count (Section 6.1.3), with `recovery_steps` timing the return to within 90% of the pre-task baseline. Depth and area are largely independent: a deep dip that snaps back in a few steps has a large depth but a small area, while a transient that stays open costs a large area at any depth. The transient also has a *shape* that no scalar captures — a sharp magnitude-driven notch or a gradual trajectory-driven bend — so we read the accuracy curves alongside the numbers; permanent retention loss, in turn, shows up in ACC/FORG rather than in either depth or area. We quote area and recovery alongside depth where they change the reading. The tables here report depth and ACC, with area where it carries the argument; the full metric set is in Appendix C, and the continual-learning metrics of Table B.2 are reported as fractions. Every entry is the mean over five seeds (three for the CIFAR generalisation block) with the seed standard deviation. With five paired seeds the smallest two-sided Wilcoxon p -value attainable is 0.0625, reached when all five seed-paired deltas share a sign; we therefore read “ $p = 0.0625$ ” as “all five seeds agree in direction” and lean on effect sizes and seed-paired consistency rather than a 0.05

threshold the design cannot reach. The full metric tables for every condition are in Appendix C.

7.1 Three-Contributor Decomposition (RQ1)

Table 7.1 reports gap depth, area, and final accuracy along the removal path $G1 \rightarrow G3 \rightarrow G4$ of Section 6.2.1, with the full-data control G2 and the NCL path-finding reference. Reading the depth column against the differencing rules of Section 6.2.1 gives the three contributor shares.

Condition	gap_depth (pp)	gap_area	ACC
G1 — Vanilla ER	19.5 ± 4.1	6.3	0.873 ± 0.005
G2 — Full-data ER	18.3 ± 2.5	8.1	0.890 ± 0.009
G3 — Balanced ER	5.7 ± 1.9	1.4	0.849 ± 0.005
G4 — Full-data balanced	1.9 ± 1.3	3.2	0.869 ± 0.002
NCL (reference)	7.2 ± 1.0	0.3	0.770 ± 0.012

Table 7.1: Decomposition conditions on rot-MNIST (momentum 0.0). NCL has the lowest bend (0.3) despite its low depth and worst accuracy — its gap is permanent forgetting, not a transient; the deep vanilla spike (19.5 pp) leaves only a modest bend (6.3) because rot-MNIST reconverges within a few steps.

The differencing of Section 6.2.1 yields

$$\begin{aligned}
 G_{\text{mag}} &= 19.5 - 5.7 = 13.8 \text{ pp} \quad (71\%), \\
 G_{\text{est}} &= 5.7 - 1.9 = 3.8 \text{ pp} \quad (19\%), \\
 G_{\text{traj}} &= 1.9 \text{ pp} \quad (10\%).
 \end{aligned}$$

The magnitude asymmetry is the dominant contributor: removing it by unit-norm balancing (G3) erases roughly two-thirds of the gap on its own. The estimator and trajectory shares are each small. The orthogonal sanity check is only partially consistent — the unbalanced full-data step removes $\text{gap_depth}(G1) - \text{gap_depth}(G2) = 1.2$ pp, against the 3.8 pp removed by full data on top of balancing. Both estimates place the estimator contribution in the low single-digit-pp range; the difference indicates that estimator noise and magnitude asymmetry are not perfectly separable, which is expected since a noisy replay gradient also inflates the first-step magnitude. The trajectory residual at G4 is 1.9 pp — small but non-zero, matching the prediction of Section 2.4 and Equation (4.3), and exactly the share the λ -curriculum is built to suppress.

The accuracy column already shows the cost structure that motivates the curriculum. Balancing (G3) reduces depth but pays 2.4 pp of final accuracy relative to vanilla ER; NCL reaches a comparable depth (7.2 pp) but sacrifices 10 pp of accuracy. A method that removes the trajectory residual without an accuracy cost is what Sections 7.2 and 7.3 test.

The bend, separated from forgetting. Depth measures the sharp notch at the boundary; `gap_area` measures the gradual *bend* that follows — the dip below the level each task recovers to (Section 6.1.3), so a permanent drop to a lower plateau does not count. Figure 7.1 shows the bend directly. A gradual dip-and-recover is visible in every condition except NCL: cleanest in the full-data runs (G2, G4), whose exact replay gradients give a smooth curve without buffer jitter, still present under vanilla ER on top of its sharp notch, and reduced to a faint sag after balancing (G3). G4 is the diagnostic case — with the magnitude spike removed its depth is only 1.9 pp, yet a bend remains (area 3.2): the trajectory residual showing up as a hanging curve rather than a spike, exactly as Section 2.4 predicts. The areas rank the bends as the eye does — $G2 > G1 > G4 > G3$ (8.1, 6.3, 3.2, 1.4). NCL is the lone exception, and the reason we reference the recovered level rather than the pre-task accuracy: a pre-referenced shortfall would call NCL the worst condition, since it sits roughly 10 pp below its starting accuracy for the rest of the task, but that shortfall is permanent forgetting — the same 10 pp it pays in final accuracy, which ACC and FORG already carry. It steps to a lower plateau and stays, with no reconvergence to integrate, so its bend is the smallest in the block (0.3). These bends are small in absolute terms because rot-MNIST reconverges within a few steps; the deeper CIFAR backbone is where the transient stays open long enough to dominate the gap (Section 7.4).

7.2 The λ -Curriculum on rot-MNIST (RQ2a, RQ2b, RQ2c)

Table 7.2 reports the full curriculum sweep at both momentum settings, against the vanilla-ER baseline (G1). We describe the momentum-off leg first, then the momentum-on leg, which is the headline

regime.

Linear curriculum (RQ2a). At momentum 0.0, gap depth falls monotonically as the ramp lengthens — $19.5 \rightarrow 17.5 \rightarrow 16.2 \rightarrow 14.3$ pp from vanilla through $N = 200$ — consistent with the $O(1/N)$ bound of Equation (4.3). The reduction is modest in absolute terms over this range of N , and it comes with a small accuracy drift: ACC falls from 0.873 (vanilla) to 0.828 at $N = 200$, a 4.4 pp cost with all five seeds agreeing in direction ($p = 0.0625$). So at momentum 0.0 the prediction that the linear curriculum matches vanilla accuracy is only partly borne out — the curriculum trades a little accuracy for depth in this regime. The same lengthening that lowers depth also stretches the tail: `recovery_steps` grows from 36 steps at $N = 50$ to 133 at $N = 200$ (Appendix C), because a slower ramp holds T_0 accuracy back longer even as it shrinks the spike. The transient bend still falls ($4.9 \rightarrow 1.8$ from $N = 50$ to $N = 200$), since the dip is shallower, but Figure 7.2 shows the longest ramp’s curve hanging visibly longer and noisier near baseline — the depth gain partly trades against a slower reconvergence.

Adaptive curriculum (RQ2b). At momentum 0.0 the adaptive schedule reaches 12.8 pp, below the best linear curriculum (14.3 pp at $N = 200$) and without an explicit ramp length N . The self-paced schedule therefore removes the knob N while at least matching the best hand-tuned linear ramp, confirming RQ2b. Its accuracy (0.833) carries the same small drift as the linear runs at this momentum setting.

The $\lambda_{\min}$ floor (RQ2c). At momentum 0.0, raising $\lambda_{\min}$ from 0 to 0.20 moves the adaptive curriculum in the predicted Pareto direction but only weakly: final accuracy edges up ($0.833 \rightarrow 0.837$) and short-horizon plasticity recovers marginally ($WP_{100} : 0.703 \rightarrow 0.710$; Appendix C), at the cost of a slight depth increase ($12.8 \rightarrow 13.9$ pp). The effect is small here because the momentum-off gap is dominated by the deterministic one-sided boundary step, which a floor on λ cannot touch. The floor becomes decisive only once momentum suppresses that step; that regime is the subject of Section 7.3 (RQ2d).

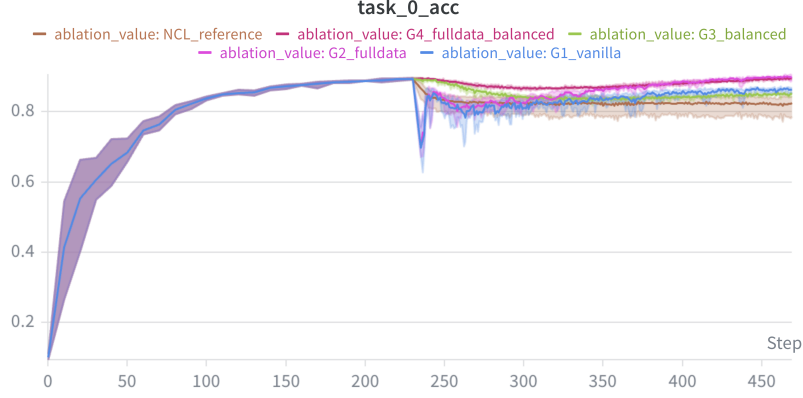


Figure 7.1: Per-step past-task (T_0) test accuracy across the $T_0 \rightarrow T_1$ transition, one curve per decomposition condition (G1 vanilla, G2 full-data, G3 balanced, G4 full-data balanced, NCL), momentum 0.0, mean over five seeds with $\pm$ std bands. The sharp boundary notch shrinks along $G1 \rightarrow G3 \rightarrow G4$, but a gradual dip-and-recover *bend* remains in every condition except NCL — clearest in the full-data curves (G2, G4) and faint after balancing. NCL (brown) instead settles at a visibly lower plateau and stays: permanent forgetting that a pre-referenced area would mistake for instability, while its transient bend is negligible.

Condition	Momentum 0.0		Momentum 0.9	
	gap_depth (pp)	ACC	gap_depth (pp)	ACC
Vanilla ER (baseline)	19.5 ± 4.1	0.873 ± 0.005	15.9 ± 2.6	0.928 ± 0.003
Linear $N=50$	17.5 ± 1.7	0.851 ± 0.023	6.4 ± 0.5	0.932 ± 0.003
Linear $N=100$	16.2 ± 4.3	0.844 ± 0.025	6.4 ± 0.6	0.932 ± 0.003
Linear $N=200$	14.3 ± 4.6	0.828 ± 0.025	6.2 ± 0.6	0.925 ± 0.007
Adaptive	12.8 ± 2.9	0.833 ± 0.025	2.5 ± 0.5	0.917 ± 0.001
Adaptive + $\lambda_{\min}=0.05$	13.2 ± 3.2	0.835 ± 0.025	2.6 ± 0.5	0.922 ± 0.001
Adaptive + $\lambda_{\min}=0.10$	13.9 ± 3.3	0.836 ± 0.026	3.0 ± 0.4	0.927 ± 0.002
Adaptive + $\lambda_{\min}=0.20$	13.9 ± 3.4	0.837 ± 0.031	3.7 ± 0.4	0.931 ± 0.002

Table 7.2: λ -curriculum sweep on rot-MNIST against the vanilla-ER baseline, at momentum 0.0 and 0.9. Depth in percentage points, ACC as a fraction; mean $\pm$ std over five seeds.

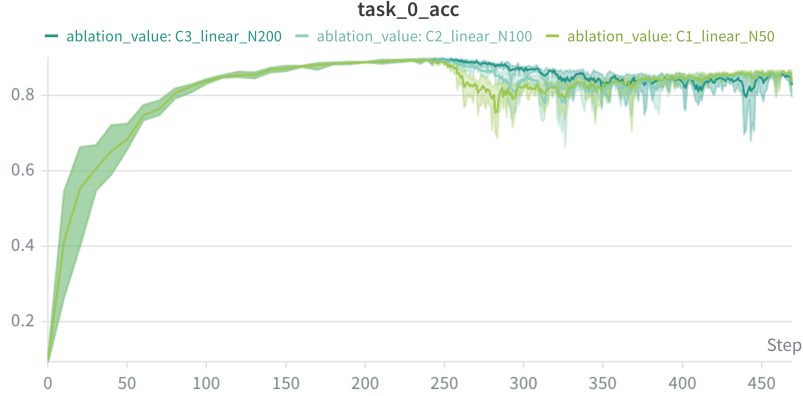


Figure 7.2: Per-step T_0 accuracy across the transition for the linear curriculum at $N \in \{50, 100, 200\}$, momentum 0.0, mean over five seeds with $\pm$ std bands. The longest ramp ($N = 200$, dark) reaches the shallowest dip but its curve hangs below baseline the longest before reconverging, matching the depth/recovery trade in the text.

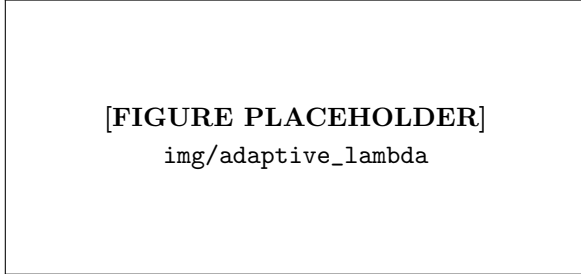


Figure 7.3: [PLACEHOLDER] The adaptive schedule in action: $\lambda(t)$ (EMA of $\|g_{\text{replay}}\|/\|g_{\text{new}}\|$) over the first 250 steps of T_1 , mean over five seeds with a $\pm$ std band, optionally overlaid with the raw gradient-magnitude ratio. Illustrates the self-paced ramp from $\lambda \approx 0$ to $\lambda \approx 1$.

7.3 Momentum Cross (RQ2d)

Two predictions sit behind RQ2d: momentum on top of the curriculum should suppress the residual stochastic oscillation, and momentum alone, without the curriculum, should not reduce gap depth. Both hold.

Momentum alone does not close the gap. For vanilla ER, momentum moves depth from 19.5 to 15.9 pp, a change in which the five seeds do not agree in direction ($p = 0.44$) and which is small next to the seed spread. The deterministic one-sided first step at the boundary is unchanged by momentum, so momentum alone leaves the gap essentially intact — the falsifiable null prediction of Section 6.2.2 is confirmed.

Momentum on top of the curriculum closes it. Applied to the curriculum, momentum has a large effect. The adaptive schedule drops from 12.8 to 2.5 pp and the linear $N = 200$ schedule from 14.3 to 6.2 pp, both with all five seeds agreeing ($p = 0.0625$). The seed-to-seed standard deviation collapses at the same time: from 2.9 to 0.5 pp for the adaptive schedule and from 4.6 to 0.6 pp for linear $N = 200$. This is the signature predicted in Section 6.2.2 — once the curriculum has made the optimum path continuous, the residual gap is stochastic oscillation around that path, and mo-

momentum averages it away (Figure 7.5). The combination of a continuous schedule and momentum, not either alone, is what drives the gap down to a few percentage points while recovering vanilla-ER accuracy.

The $\lambda_{\min}$ floor tunes the residual trade.

With momentum on, the floor that barely registered at momentum 0.0 becomes a clean Pareto knob. Its mechanism is visible on the plasticity side (Figure 7.4): a higher floor lets the new task train from the first post-switch step, so T_1 accuracy climbs faster the larger $\lambda_{\min}$ is. Raising $\lambda_{\min}$ from 0 to 0.20 lifts gap depth slightly ($2.5 \rightarrow 3.7$ pp) while restoring final accuracy ($0.917 \rightarrow 0.931$) and short-horizon plasticity ($WP_{100} : 0.798 \rightarrow 0.812$; Appendix C). At $\lambda_{\min} = 0.20$ the curriculum matches vanilla-ER accuracy (0.931 vs 0.928) and the best linear curricula (0.932) while holding depth to 3.7 pp — roughly half the linear curricula’s 6.4 pp and far below vanilla’s 15.9 pp. This is the setting carried forward to CIFAR (Section 7.4).

7.4 Generalisation: CIFAR-10 (RQ3)

The CIFAR-10 block tests whether the rot-MNIST finding carries to a deeper backbone (ResNet-18), a stronger task shift (clean $\rightarrow$ corruption), and the two normalisation choices that govern the post-switch recovery (batch norm, group norm). Table 7.3 reports the six headline conditions; the shipped curriculum is the adaptive schedule with $\lambda_{\min} = 0.20$.

The rot-MNIST result carries over and strengthens. Under batch norm the adaptive curriculum cuts depth from 11.8 pp (vanilla) to 4.8 pp; under group norm, where vanilla ER suffers a catastrophic 41.2 pp drop, it cuts depth to 5.2 pp. In both cases final accuracy is unchanged (≤ 1 pp difference from vanilla) while worst-case retention improves sharply — min-ACC rises from 0.309 to 0.670 under group norm.

Here the transient is not a gentle bend but a deep stability gap that stays open (Figure 7.6): under group norm vanilla ER crashes T_0 accuracy from ~ 0.73 almost to chance and then takes several hundred steps to recover. Because the ResNet re-converges far more slowly than the MLP, that gap

costs much more accuracy over time — its area is large because it stays open, not because the shape is exotic. (Area is depth $\times$ duration in steps, and the CIFAR window is several times longer than rot-MNIST’s, so we compare areas only within this block.) Vanilla ER has both the deepest spike and the largest area under group norm (41.2 pp, 34.9), with batch norm milder on both (11.8 pp, 17.5); NCL is worse still (55.1 and 52.6, the largest in the block), mirroring its depth regression. The adaptive curriculum closes the gap an order of magnitude faster, dropping the area to 3.6 (batch norm) and 1.7 (group norm): the past task barely leaves its plateau through the switch. The NCL contrast is the cleanest separation in the study between acting in schedule space (the curriculum) and acting in parameter space (NCL): on this deeper backbone NCL’s path-finding preconditioner makes the gap *larger* than vanilla ER (25.8 pp depth under batch norm, the opposite of its rot-MNIST behaviour), while the curriculum’s benefit is robust to depth and normalisation.

Generalisation across corruptions and sequence length. The generalisation block (three seeds, group-norm ship config) pairs the curriculum against a vanilla-ER control on shifts the method was not tuned on (Table 7.4). On the two-task novel corruptions the curriculum generalises cleanly: depth falls from 46.3 to 7.3 pp on shot noise and from 8.4 to 3.9 pp on contrast. The three-task sequence is the exception — there the curriculum underperforms its vanilla control (32.8 vs 4.4 pp, with a large seed variance of ± 14.5 pp), an anomaly that does not appear in the batch-norm variant of the same sequence (Appendix C). We read this as the adaptive schedule interacting poorly with several closely spaced group-norm transitions rather than a single one, and flag it as a boundary of the method’s reliability rather than evidence against the two-task result; final accuracy stays comparable to vanilla throughout.

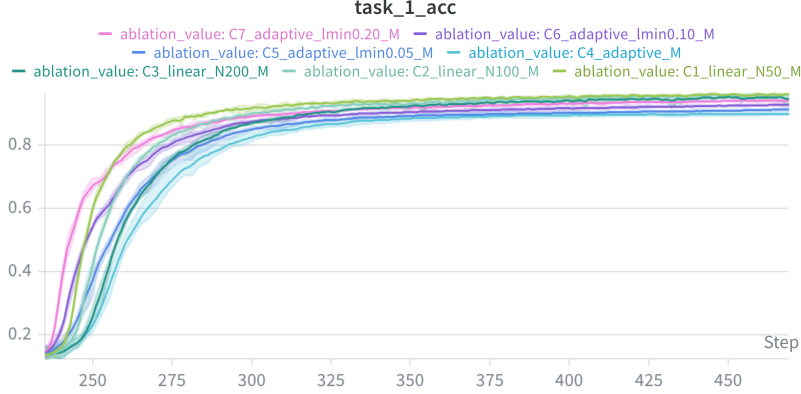


Figure 7.4: New-task (T_1) accuracy over the post-switch window for the full curriculum sweep at momentum 0.9, mean over five seeds with $\pm$ std bands. The $\lambda_{\min}$ floor controls early plasticity: the highest floor (C7, $\lambda_{\min} = 0.20$) learns T_1 fastest, the unfloored adaptive schedule (C4) slowest, with the linear ramps in between. This is the plasticity that the floor buys back, at the small depth cost reported in Table 7.2.

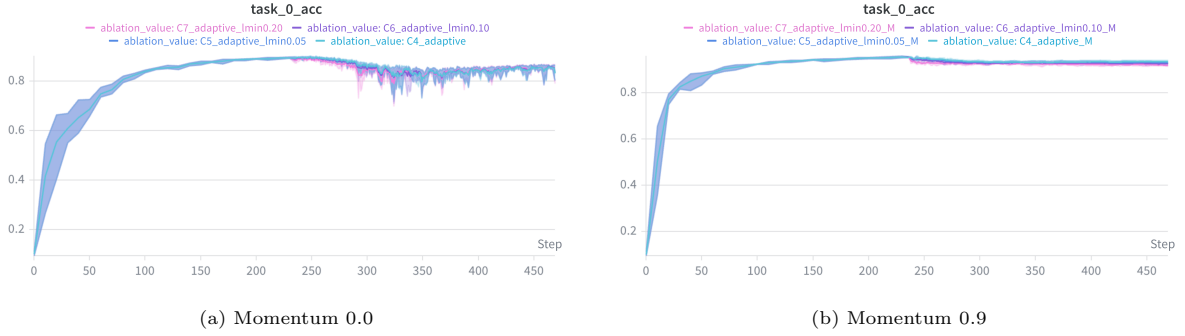


Figure 7.5: The adaptive curriculum (with $\lambda_{\min} \in \{0, 0.05, 0.10, 0.20\}$) on T_0 accuracy across the transition, at momentum 0.0 (left) versus 0.9 (right), mean over five seeds with $\pm$ std bands. Momentum does not just shrink the dip: it removes the post-switch oscillation that makes the momentum-0.0 curves hang and jitter below baseline, leaving a shallow, fast reconvergence. This is the stochastic residual the curriculum alone cannot average away.

Norm	Method	gap_depth (pp)	gap_area	ACC	min-ACC
BatchNorm	D1 — Vanilla ER	11.8 ± 2.5	17.5	0.723 ± 0.008	0.644 ± 0.023
	D2 — NCL	25.8 ± 5.2	55.1	0.710 ± 0.005	0.504 ± 0.045
	D3 — Adaptive $\lambda_{\min}=0.20$	4.8 ± 0.7	3.6	0.722 ± 0.011	0.713 ± 0.017
GroupNorm	D4 — Vanilla ER	41.2 ± 13.5	34.9	0.730 ± 0.011	0.309 ± 0.111
	D5 — NCL	29.7 ± 10.0	52.6	0.711 ± 0.017	0.423 ± 0.083
	D6 — Adaptive $\lambda_{\min}=0.20$	5.2 ± 2.1	1.7	0.731 ± 0.020	0.670 ± 0.055

Table 7.3: CIFAR-10 headline block (ResNet-18, momentum 0.9, buffer 5,000, three methods $\times$ two normalisations, five seeds). The adaptive curriculum cuts both gap depth and its area (gap_area, how much accuracy the gap costs before it recovers) far below vanilla ER and NCL, at matched final accuracy and much higher worst-case retention (min-ACC).

Shift (GroupNorm)	Ship depth	Vanilla depth
2-task shot noise	7.3 ± 5.9	46.3 ± 12.0
2-task contrast	3.9 ± 3.1	8.4 ± 4.4
3-task g.n.+shot	32.8 ± 14.5	4.4 ± 0.4

Table 7.4: CIFAR-10 generalisation block, gap depth (pp), group-norm ship config versus vanilla-ER control (three seeds). The curriculum wins on both two-task novel corruptions; the three-task group-norm sequence is an outlier (see text).

7.5 Long-Sequence rot-MNIST

The two-task design isolates a single transition. To check that the curriculum holds across several boundaries, we ran a five-task rot-MNIST sequence (rotations $[0, 30, 60, 90, 120]^\circ$, four transitions) at momentum 0.9 (Table 7.5).

Condition	gap_depth (pp)	ACC
L1 — Vanilla ER	8.9 ± 1.7	0.889 ± 0.007
L2 — NCL	2.5 ± 0.6	0.815 ± 0.003
L3 — Adaptive $\lambda_{\min}=0.00$	4.4 ± 0.2	0.896 ± 0.003
L4 — Adaptive $\lambda_{\min}=0.10$	4.5 ± 0.6	0.896 ± 0.003

Table 7.5: Five-task rot-MNIST (four transitions, momentum 0.9). Gap depth is the mean worst drop across the four boundaries. The adaptive curriculum halves the vanilla-ER gap while preserving accuracy; NCL reaches the lowest depth but pays ~ 8 pp of accuracy.

The adaptive curriculum carries past a single transition: it roughly halves the vanilla-ER gap ($8.9 \rightarrow 4.4$ pp) across four boundaries while slightly *improving* final accuracy ($0.889 \rightarrow 0.896$). NCL reaches the lowest depth of the four (2.5 pp) — consistent with Kao et al. [14] reporting its advantage at higher task counts — but at a large accuracy cost (0.815, 8 pp below the curriculum). The $\lambda_{\min}$ floor makes little difference to depth here. As on the two-task MLP, the per-boundary bends are small for every condition ($\text{gap_area} \leq 1.5$; Appendix C) — rot-MNIST reconverges quickly after each switch — so what accumulates across the sequence is permanent forgetting, not a hanging transient. Figure 7.7 shows this directly: vanilla ER (brown) steps down at each of the four boundaries and holds the lowest plateau, while the adaptive curricula keep a higher plateau throughout; NCL stays flattest but lowest,

the accuracy cost again. The curriculum thus keeps the property that matters across multiple transitions: a low gap at no accuracy cost.

7.6 Summary of Findings

Across blocks, four results stand out. (i) On rot-MNIST the stability gap is dominated by the first-step magnitude asymmetry (71%), with a small trajectory residual (10%) that survives exact gradients and balancing — the residual the curriculum targets. (ii) The adaptive λ -curriculum with momentum reduces gap depth from 15.9 to 3.7 pp at matched accuracy, where momentum alone does nothing and the curriculum alone leaves a stochastic residual: the two are complementary, as predicted. (iii) The result carries to a ResNet-18 on CIFAR-10 under both normalisations and to novel two-task corruptions, where the curriculum beats both vanilla ER and NCL on depth and worst-case retention at matched accuracy. (iv) The known boundaries are an accuracy drift for the linear curriculum at momentum 0.0, and a high-variance failure of the adaptive schedule on a three-task group-norm sequence. Underlying all four is a distinction the depth headline can hide: how deep the gap goes and how much accuracy it costs before recovering are separate axes. The decomposition shows the second one directly — once the magnitude spike is removed, the trajectory residual remains as a gradual *bend* (G4, depth 1.9 pp but a visible area), the hanging curve the theory predicts. On the shallow MLP that transient closes in a few steps and costs little; on the deeper CIFAR ResNet it stays open for hundreds of steps and dominates the gap, and the curriculum’s largest wins are there. Throughout, the area is referenced against the recovered level so that permanent forgetting — NCL’s apparent instability — is not mistaken for a transient. The discussion interprets these against the theory of Sections 3 and 4.

8 Discussion

The results let us return to the claim the thesis set out to defend: that in its dominant components the stability gap is a symptom of the task-boundary discontinuity, and that removing the discontinuity is a better intervention than navigating it. The de-

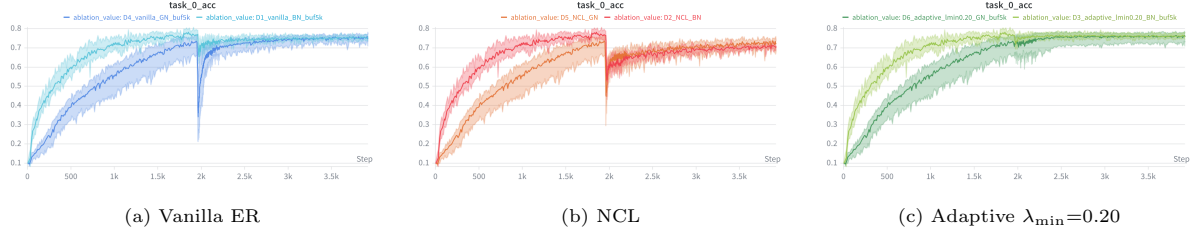


Figure 7.6: Per-step T_0 accuracy across the CIFAR-10 transition (ResNet-18, momentum 0.9, buffer 5,000), grouped by method; each panel overlays batch norm and group norm, mean over five seeds with $\pm$ std bands. Vanilla ER (a) dips shallowly but reconverges slowly under batch norm and crashes deep under group norm — a stability gap that stays open a long time either way. NCL (b) leaves a deep, slow gap under both norms. The adaptive curriculum (c) holds T_0 accuracy almost flat through the switch, closing the gap fast.

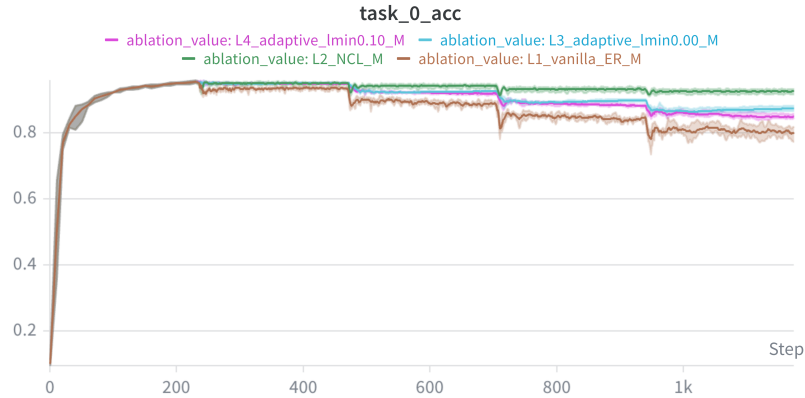


Figure 7.7: First-task (T_0) accuracy across the full five-task rot-MNIST sequence (rotations $[0, 30, 60, 90, 120]^\circ$, four transitions, momentum 0.9), mean over five seeds with $\pm$ std bands. Vanilla ER (brown) steps down furthest at every boundary and holds the lowest plateau — permanent forgetting, the per-boundary transient bends being small; the adaptive curricula (pink, cyan) hold a higher plateau; NCL (green) stays flattest but lowest.

composition supports the first half of the claim, the curriculum-and-momentum results the second. What the numbers also force is a sharper reading of what “the gap” is — not one quantity but two, a depth and a hanging tail, that the theory predicts should respond to different parts of the fix and that come apart most clearly on the deeper backbone.

8.1 The boundary is the cause, and it has two faces

The decomposition (RQ1) places roughly two-thirds of the rot-MNIST gap in the first-step magnitude asymmetry and a further fifth in estimator and trajectory terms, with the trajectory residual surviving exact gradients and unit-norm balancing. This is the pattern the discontinuity account predicts: the part of the gap that vanishes when the new-task gradient is introduced gradually (G_{mag} , G_{traj}) is the part tied to the boundary, and the part that does not (G_{est}) is the buffer’s, independent of the schedule. That the trajectory residual is small on the MLP is not evidence against it; it is what Equation (4.3) expects on a network that reconverges in a few steps, where the $O(1/N)$ tail has little time to accumulate.

The residual matters because it is the half of the gap that the depth headline hides. Depth measures the instantaneous notch the one-sided first step cuts; `gap_area` measures the bend that follows, the cost the transient carries before the curve reconverges. The G4 condition isolates this directly: with the magnitude spike removed its depth is only 1.9 pp, yet a visible bend remains — the trajectory residual showing up as a hanging curve rather than a spike, exactly the shape the theory attaches to the $1/N$ term. On rot-MNIST this bend is cheap. On the CIFAR ResNet, which reconverges over hundreds of steps rather than a few, the same transient stays open long after the initial notch, so most of the accuracy the gap costs comes from the prolonged recovery rather than the depth of the dip — and that recovery cost is where the curriculum’s margin over the baselines is largest. The depth-versus-area distinction is therefore not bookkeeping: it is the axis along which the method’s value scales with how slowly a network recovers, and a depth-only reading would have understated the result on exactly the architectures where it counts.

The same distinction guards against a false nega-

tive in the other direction. NCL reaches a low depth on rot-MNIST but its curve steps to a lower plateau and stays there, which a gap metric referenced to the pre-task accuracy would score as severe instability. Referenced instead to the level each task recovers to, that shortfall is correctly read as permanent forgetting — the same loss ACC and FORG already report — and not as a transient the curriculum could close. The gap and forgetting are different failures, and keeping them apart is what lets the decomposition attribute the gap to the boundary rather than to capacity loss.

8.2 Why shaping beats path-finding

The cleanest separation in the study is the contrast between NCL and the curriculum on the deeper backbone. The two interventions target the same discontinuity from opposite sides: NCL accepts the jump and preconditions each update with past-task curvature (path-finding); the curriculum removes the jump and leaves the geometry of each step to SGD (landscape-shaping). On rot-MNIST both lower depth, so the two are hard to tell apart. On CIFAR they diverge sharply — NCL’s preconditioner makes the gap *larger* than vanilla ER under batch norm, the opposite of its rot-MNIST behaviour, while the curriculum’s benefit holds across both normalisations and both backbones.

This is the failure mode the motivation anticipated for path-finding (Section 3.2). A preconditioner rescales the gradient, and on a deeper network the curvature estimate it depends on is harder to get right and its rescaling interferes with the step-size adaptation SGD relies on; when the estimate is off, steering across the discontinuity can steer badly. The curriculum changes only a scalar loss weight and leaves the step size untouched, so it has nothing analogous to mis-estimate. The result is the practical case for the thesis’s methodological stance: an intervention that removes the problematic structure degrades more gracefully with depth than one that builds machinery to cope with it. We do not claim path-finding fails in general — NCL reaches the lowest depth of any method on the long rot-MNIST sequence, consistent with Kao et al. [14] — only that it pays an accuracy cost the curriculum does not and that its advantage did not survive the move to a deeper backbone.

8.3 Two mechanisms, two parts of the fix

The momentum cross (RQ2d) is the most direct test of the theory’s mechanism, and it confirms a falsifiable null: momentum alone, without the curriculum, leaves the gap essentially intact, with seeds disagreeing on even the sign of the small change. This is expected, because the first step at the boundary is deterministic — the new-task gradient against a near-zero replay gradient — and averaging cannot soften a step that has no stochastic component to average. Momentum becomes decisive only once the curriculum has made the optimum path continuous, at which point the residual gap is the stochastic oscillation around that path and momentum averages it away, collapsing both the depth and its seed-to-seed variance.

Read against Equation (4.3), the curriculum and momentum address different terms. The curriculum lengthens the ramp, which shrinks the deterministic curvature spike ($1/N^2$, the depth) and the trajectory lag ($1/N$, the area) but cannot remove the per-step jitter the continuous-limit argument abstracts away. Momentum removes that jitter but does nothing to the deterministic step. Neither alone closes the gap to a few points at matched accuracy; together they do. The $\lambda_{\min}$ floor then trades a small slice of depth back for early plasticity, and only becomes a clean Pareto knob once momentum has suppressed the deterministic step the floor cannot touch — which is why it barely registered at momentum 0 and tuned a real trade-off at momentum 0.9. The three refinements are not interchangeable knobs; each acts on a part of the gap the others leave alone, in the order the theory predicts.

8.4 Relation to blurry task boundaries

The curriculum and the blurry-boundary stream of Section 2.5 descend the same moving mixture of old- and new-task gradients, but they differ in what they hold fixed, and that difference is what makes the curriculum the cleaner test of the mechanism. A blurry stream changes the data distribution and the loss composition together: as T_1 samples enter the batch the gradient reweights, but the set of examples the model trains on also changes, so a reduc-

tion in forgetting cannot be attributed to the gradual reweighting alone. A blurry stream also carries no lasting retention pressure — once the transition completes and the batch is entirely T_1 , the old-task data leaves the stream and nothing anchors the model to the old optimum, so forgetting can resume. The λ -curriculum leaves data sampling and buffer contents untouched, varies only the weight on the current-task loss, and retains an explicit replay term after the new task has fully ramped in. It therefore isolates the reweighting as the one variable responsible for any change in the gap and sustains the retention signal a blurry stream loses. Our result supports the blurry-boundary finding that softer transitions forget less [16], while separating the causal variables that line of work leaves entangled.

The result also rests on the low-loss corridor that mode connectivity establishes (Section 2.6). The curriculum can only keep the optimiser on a continuous path from θ_A^* to θ_{joint}^* if such a path exists; Mirzadeh et al. [21] show that in stable training regimes the sequential and joint solutions lie in the same linearly connected basin. The curriculum’s job is to hold the iterate on that corridor rather than let it bow away, and the trajectory residual it leaves is the small lag with which a finite-rate ramp tracks the corridor’s moving floor.

8.5 Limitations

Three limits bound these claims. The first is statistical: five seeds put a floor of $p = 0.0625$ on the two-sided Wilcoxon test, so we lean on effect sizes and seed-paired direction rather than a 0.05 threshold the design cannot reach. The effects we rest on — the order-of-magnitude depth reductions on CIFAR, the variance collapse under momentum — are large relative to the seed spread, but the smaller ones, including the $\lambda_{\min}$ trade-offs, are read as directional consistency, not significance.

The second is the accuracy drift of the linear curriculum at momentum 0, where lengthening the ramp lowers depth but costs several points of final accuracy. The prediction that the curriculum matches vanilla accuracy holds in the momentum-on regime that the method ships in, but not at momentum 0, where a slow ramp holds the new task back long enough to leave a mark on final accuracy. The method’s accuracy-neutrality is therefore

a property of the full curriculum-plus-momentum configuration, not of the ramp alone.

The third is the three-task group-norm sequence, where the adaptive schedule underperformed its vanilla control with high seed variance, an anomaly absent from the batch-norm variant of the same sequence. We read this as the adaptive gradient-ratio signal interacting poorly with several closely spaced group-norm transitions rather than a single isolated one, and flag it as a boundary of the method’s reliability. More broadly, the headline analysis is built on a single-transition, two-task design chosen to isolate one boundary cleanly; the long-sequence checks suggest the property carries, but a systematic study across many boundaries, longer task sequences, and class-incremental rather than domain-incremental shifts is left open.

8.6 Implications and outlook

The practical reading is narrow but concrete. For continual-learning systems that are queried during training rather than only at task boundaries, the cost of the gap is the accuracy lost while the transient is open, and that cost grows with how slowly the network reconverges. The curriculum addresses exactly this cost, at almost no final-accuracy penalty and at negligible overhead — a single changing scalar, no stored curvature, no gradient projection — which makes it cheap to add on top of an existing replay pipeline.

Two directions follow from the limitations. The first is the multi-transition regime where the adaptive schedule showed strain: an adaptive signal robust to closely spaced boundaries, perhaps one that conditions on a boundary detector rather than on the raw gradient ratio, would extend the method past its current reliable range. The second is the scope of the bound itself, which is stated for the trajectory contributor and weakens once the new task’s curvature overpowers the old (Section 4.1); characterising where that crossover sits, and whether the magnitude tempering the curriculum also provides admits a guarantee of its own, would narrow the distance between what the theory proves about the trajectory term and what the experiments show the full method does.

References

- [1] Rahaf Aljundi, Min Lin, Baptiste Goujaud, and Yoshua Bengio. Gradient based sample selection for online continual learning, 2019. URL <https://arxiv.org/abs/1903.08671>. 5, 8, 13
- [2] Jihwan Bang, Heesu Kim, YoungJoon Yoo, Jung-Woo Ha, and Jonghyun Choi. Rainbow memory: Continual learning with a memory of diverse samples, 2021. URL <https://arxiv.org/abs/2103.17230>. 7
- [3] Arslan Chaudhry, Marc’Aurelio Ranzato, Marcus Rohrbach, and Mohamed Elhoseiny. Efficient lifelong learning with a-gem, 2019. URL <https://arxiv.org/abs/1812.00420>. 4, 6
- [4] Arslan Chaudhry, Marcus Rohrbach, Mohamed Elhoseiny, Thalaiyasingam Ajanthan, Puneet K. Dokania, Philip H. S. Torr, and Marc’Aurelio Ranzato. On tiny episodic memories in continual learning, 2019. URL <https://arxiv.org/abs/1902.10486>. 5
- [5] Felix Draxler, Kambis Veschgini, Manfred Salmhofer, and Fred A. Hamprecht. Essentially no barriers in neural network energy landscape, 2019. URL <https://arxiv.org/abs/1803.00885>. 4, 7
- [6] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel M. Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis, 2020. URL <https://arxiv.org/abs/1912.05671>. 7
- [7] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry Vetrov, and Andrew Gordon Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns, 2018. URL <https://arxiv.org/abs/1802.10026>. 4, 7
- [8] Yiduo Guo, Jie Fu, Huishuai Zhang, Dongyan Zhao, and Yikang Shen. Efficient continual pre-training by mitigating the stability gap, 2024. URL <https://arxiv.org/abs/2406.14833>. 6

- [9] Yunhui Guo, Mingrui Liu, Tianbao Yang, and Tajana Rosing. Improved schemes for episodic memory-based lifelong learning, 2020. URL <https://arxiv.org/abs/1909.11763>. 6
- [10] Md Yousuf Harun and Christopher Kanan. Overcoming the stability gap in continual learning, 2024. URL <https://arxiv.org/abs/2306.01904>. 6
- [11] Dan Hendrycks and Thomas Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://arxiv.org/abs/1903.12261>. 31
- [12] Timm Hess, Tinne Tuytelaars, and Gido M. van de Ven. Two complementary perspectives to continual learning: Ask not only what to optimize, but also how, 2024. URL <https://arxiv.org/abs/2311.04898>. 3, 6
- [13] Sandesh Kamath, Albin Soutif-Cormerais, Joost van de Weijer, and Bogdan Raducanu. The expanding scope of the stability gap: Unveiling its presence in joint incremental learning of homogeneous tasks, 2024. URL <https://arxiv.org/abs/2406.05114>. 6
- [14] Ta-Chu Kao, Kristopher T. Jensen, Gido M. van de Ven, Alberto Bernacchia, and Guillaume Hennequin. Natural continual learning: success is a journey, not (just) a destination, 2021. URL <https://arxiv.org/abs/2106.08085>. 3, 4, 6, 8, 14, 15, 16, 24, 26, 31
- [15] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, March 2017. ISSN 1091-6490. doi: 10.1073/pnas.1611835114. URL <http://dx.doi.org/10.1073/pnas.1611835114>. 4, 5, 6, 8
- [16] Hyunseo Koh, Dahyun Kim, Jung-Woo Ha, and Jonghyun Choi. Online continual learning on class incremental blurry task configuration with anytime inference, 2022. URL <https://arxiv.org/abs/2110.10031>. 7, 27
- [17] Matthias De Lange, Gido van de Ven, and Tinne Tuytelaars. Continual evaluation for lifelong learning: Identifying the stability gap, 2023. URL <https://arxiv.org/abs/2205.13452>. 3, 4, 6, 8, 11, 13, 33
- [18] David Lopez-Paz and Marc’Aurelio Ranzato. Gradient episodic memory for continual learning, 2022. URL <https://arxiv.org/abs/1706.08840>. 4, 6, 8
- [19] Michael McCloskey and Neil J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. *The Psychology of Learning and Motivation*, 24:104–169, 1989. 3
- [20] Martial Mermillod, Aurélia Bugaïska, and Patrick Bonin. The stability-plasticity dilemma: Investigating the continuum from catastrophic forgetting to age-limited learning effects. *Frontiers in psychology*, 4:504, 08 2013. doi: 10.3389/fpsyg.2013.00504. 3
- [21] Seyed Iman Mirzadeh, Mehrdad Farajtabar, Dilan Gorur, Razvan Pascanu, and Hassan Ghasemzadeh. Linear mode connectivity in multitask and continual learning, 2020. URL <https://arxiv.org/abs/2010.04495>. 7, 27
- [22] Roger Ratcliff. Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97:285–308, 04 1990. doi: 10.1037/0033-295X.97.2.285. 3
- [23] Shagun Sodhani, Mojtaba Faramarzi, Saniket Vaibhav Mehta, Pranshu Malviya, Mohamed Abdelsalam, Janarthanan Janarthanan, and Sarath Chandar. An introduction to lifelong supervised learning, 2022. URL <https://arxiv.org/abs/2207.04354>. 6
- [24] Albin Soutif-Cormerais, Antonio Carta, and Joost Van de Weijer. Improving online continual learning performance and stability with temporal ensembles, 2023. URL <https://arxiv.org/abs/2306.16817>. 6

- [25] Gido M. van de Ven, Nicholas Soures, and Dhireesha Kudithipudi. *Continual learning and catastrophic forgetting*, page 153–168. Elsevier, 2025. ISBN 9780443157554. doi: 10.1016/b978-0-443-15754-7.00073-0. URL <http://dx.doi.org/10.1016/B978-0-443-15754-7.00073-0>. 3, 5
- [26] Jeffrey S. Vitter. Random sampling with a reservoir. *ACM Trans. Math. Softw.*, 11(1): 37–57, March 1985. ISSN 0098-3500. doi: 10.1145/3147.3165. URL <https://doi.org/10.1145/3147.3165>. 5, 31
- [27] Wojciech Łapacz, Daniel Marczak, Filip Szatkowski, and Tomasz Trzciński. Exploring the stability gap in continual learning: The role of the classification head, 2024. URL <https://arxiv.org/abs/2411.04723>. 6

A Implementation Details

This appendix records the implementation specifics that the methods of Section 6 depend on but that would interrupt the argument in the main text: the experience-replay variants, and the NCL realisation together with the hyperparameters fixed by the rot-MNIST tuning sweep.

A.1 Datasets

rot-MNIST rotates each 28×28 image by the per-task angle and feeds the flattened greyscale vector to the MLP. The CIFAR-10 domain shifts are generated with the `imagecorruptions` package, which implements the standard common-corruptions benchmark of Hendrycks and Dietterich [11]; the Gaussian noise, shot noise, and contrast shifts are applied at severity 3 on the benchmark’s 1–5 scale, per sample at load time, followed by the standard CIFAR-10 channel normalisation. The clean task (“none”) applies the normalisation only.

A.2 Experience replay

The replay buffer uses reservoir sampling [26], so the stored set is a uniform sample of the past stream at any budget. Three modes appear in the experiments:

- *Standard ER* (the G1 and all C-series conditions): each step takes one combined backward pass on $L_{\text{current}} + L_{\text{replay}}$, with half the mini-batch drawn from the current task and half sampled with replacement from the buffer.
- *Balanced ER* (G3): two separate backward passes whose gradients are individually unit-normalised before being combined 50/50, removing the magnitude asymmetry between g_{new} and g_{replay} .
- *Full-data replay* (G2, G4): the replay gradient is computed on *every* stored sample exactly once per step (no sub-sampling). Paired with a buffer equal to the full past-task training set (60,000 for rot-MNIST T_0), this yields the deterministic empirical past-task gradient at the current parameters, removing estimator variance.

The λ -curriculum re-weights only the current-task term, minimising $L_\lambda = L_{\text{replay}} + \lambda(t) L_{\text{current}}$; the adaptive schedule reads $\|g_{\text{replay}}\|/\|g_{\text{new}}\|$ through an exponential moving average ($\alpha = 0.05$) and applies the optional floor λ_{min} .

A.3 Natural Continual Learning

NCL follows Kao et al. [14] (their Eq. 8 and Algorithm 1). The update preconditions the new-task gradient by the prior covariance Λ_{k-1}^{-1} and adds the Bayesian restoring term,

$$\theta \leftarrow \theta - \eta \left[\Lambda_{k-1}^{-1} \nabla L_k(\theta) + (\theta - \mu_{k-1}) \right],$$

with Λ_{k-1} summarised by a K-FAC factorisation $F \approx A \otimes G$ per linear layer and an identity prior $p_w = \alpha I$. The trust radius of the paper’s Eq. 7 is absorbed into the learning rate; we log a `tr_scale` diagnostic but do not clip the step.

Tuned hyperparameters. The values used in every NCL condition are those that won the rot-MNIST tuning sweep below (Table A.1). The paper-literal prior $\alpha = 1.0$ over-regularises on rot-MNIST and the damping ε is a numerical safeguard only — the identity prior, not the damping, is what bounds Λ^{-1} .

Hyperparameter	Symbol	Value
Prior precision	α (<code>prior_init</code>)	0.1
Damping	ε (<code>damping</code>)	1e-3
Fisher samples (K-FAC)	<code>fisher_samples</code>	1,000
Trust radius (diagnostic)	r (<code>trust_radius</code>)	1.0
Learning rate	η	0.1
Momentum	—	0.9

Table A.1: NCL hyperparameters fixed for all conditions, from the rot-MNIST tuning sweep of Table A.2.

Tuning sweep. The prior precision α was tuned on the two-task rot-MNIST/MLP setting (Table A.2), holding damping at 1e-3 and `fisher_samples` at 1,000. Smaller α improves over the paper-literal $\alpha = 1.0$, but the gain saturates: $\alpha = 0.1$ gives the best stability-accuracy trade-off, while $\alpha \leq 0.03$ becomes unstable at $\eta = 0.1$ (the natural gradient amplifies the raw gradient by up to $1/\alpha$ in any direction), and halving the learning rate does not recover it.

Config	α	η	Momentum
alpha_0.1 (chosen)	0.1	0.1	0.9
alpha_0.03	0.03	0.1	0.9
alpha_0.01	0.01	0.1	0.9
alpha_0.003	0.003	0.1	0.9
alpha_0.03_lowlr	0.03	0.05	0.9

Table A.2: NCL prior-precision tuning sweep on two-task rot-MNIST. Damping (1e-3) and `fisher_samples` (1,000) held fixed.

B Metric Definitions

This appendix gives the full definitions of every metric recorded by the evaluation pipeline of Section 6.1.3. The main text explains the measures the argument turns on; the tables below are the look-up reference for the complete set.

Stability-gap metrics. Computed from the dense per-step evaluation. Write a_j^{pre} for the pre-task accuracy on task j and $a_j(t)$ for its accuracy at step t of the new task.

Metric	Definition	Role
gap_depth	$\max_j (a_j^{\text{pre}} - \min_t a_j(t))$ over the full record window	Primary metric — worst single point of instability
gap_area	$\sum_j \int_0^T \max(0, b_j - a_j(t)) dt$, trapezoidal over the full new-task record (to task end T), with $b_j = \min(a_j^{\text{pre}}, \bar{a}_j^{\text{end}})$ and $\bar{a}_j^{\text{end}}$ the trailing-window mean (last $\max(5, \lceil 0.1n \rceil)$ samples); accuracy at or above b_j contributes 0	Transient <i>bend</i> : dip below the recovered level, excluding permanent forgetting and continued learning (units: accuracy · steps)
max_drop	gap_depth restricted to the first 200 steps	Fixed-window metric, kept for back-comparability
recovery_steps	Given a record has dipped below the threshold, the first step — counted from the transition — at which <i>every</i> past task is simultaneously at $\geq 90\%$ of its pre-task baseline	None if no sub-threshold dip occurs, or if recovery is not reached within the record

Table B.1: Stability-gap metrics, recorded from dense per-step evaluation. **gap_depth** is the headline measure used in every contrast.

Continual-learning metrics. Following Lange et al. [17], computed at task boundaries and over the full record. Let $A(E_j, f_n)$ be the accuracy on the test set of task j evaluated at the model f_n after training stage n , and let N be the number of tasks. The full task-boundary accuracy matrix $R[i, j] = A(E_j, f_{T_i})$ is logged for every run, so any downstream metric can be reconstructed.

Metric	Definition	Role
ACC	$\frac{1}{N} \sum_j A(E_j, f_{T_{N-1}})$	Average task accuracy at the final model
FORG	$\frac{1}{N-1} \sum_{i < N-1} [A(E_i, f_{T_i}) - A(E_i, f_{T_{N-1}})]$	Average drop from right-after-learning to final
min-ACC	$\frac{1}{N-1} \sum_{i < N-1} \min\{A(E_i, f_n) : n > T_i\}$	Worst-case retention since learning
WF _w	Mean over tasks of the largest accuracy drop in any window of w consecutive evaluations ($w \in \{10, 100\}$)	Worst-case stability over short / medium horizons
WP _w	Symmetric counterpart of WF _w : the largest accuracy gain ($w \in \{10, 100\}$)	Plasticity / learning velocity
WC-ACC	$\frac{1}{N} A(E_{N-1}, f_{T_{N-1}}) + (1 - \frac{1}{N}) \text{min-ACC}$	Worst-case trade-off — a lower bound on ACC

Table B.2: Continual-learning metrics of Lange et al. [17], recorded at task boundaries and over the full record.

Gradient-fidelity diagnostics. Recorded at the dense cadence for the decomposition conditions only. The tracker compares the replay-buffer gradient g_{replay} against the true past-task gradient g_{true} , computed at every step by a separate forward and backward pass over the entire past-task training set (no sub-sampling, so g_{true} is the deterministic empirical past-task gradient at the current parameters).

Diagnostic	Logged name	Role
$\cos(g_{\text{replay}}, g_{\text{true}})$	<code>true_grad_cosine</code>	Buffer-side directional fidelity — the estimator-bias contributor of Section 6.2.1
$\ g_{\text{replay}}\ /\ g_{\text{true}}\ $	<code>true_grad_mag_ratio</code>	Buffer-side magnitude fidelity
$\ g_{\text{new}}\ /\ g_{\text{replay}}\ $	<code>grad_ratio</code>	Update-side magnitude asymmetry — large at step 1 when $\ g_{\text{replay}}\ \approx 0$

Table B.3: Gradient-fidelity diagnostics, recorded over the new task’s first ~ 500 steps. Each is logged as a per-step series together with its mean (and minimum, for the cosine). The first two carry the `true_grad` prefix because they are measured against g_{true} ; `grad_ratio` compares g_{replay} to the current-task gradient g_{new} and its inverse drives the adaptive curriculum of Section 6.2.2.

C Complete Results

This appendix gives the full metric set for every condition, as recorded by the aggregation pipeline of Section 6.1.3. Each row is the mean over five seeds (three for the CIFAR generalisation block) with the seed standard deviation. Accuracy-type metrics (ACC, FORG, min-ACC, WF_{100} , WP_{100} , WC-ACC) are fractions; `gap_depth` is in percentage points (pp); `gap_area` is the end-referenced transient bend (accuracy-fraction $\times$ steps, Table B.1) — the dip below the level each task recovers to, so permanent forgetting does not contribute; “Recov.” is `recovery_steps`, with “—” marking conditions where every past task stayed above the 90% recovery threshold for the whole window (no recovery event to time).

C.1 rot-MNIST: Decomposition (G-series)

Condition	ACC	FORG	min-ACC	WF_{100}	WP_{100}	WC-ACC	Depth	Area	Recov.
<i>Momentum 0.0</i>									
G1 vanilla	0.873 ± 0.005	0.019 ± 0.020	0.706 ± 0.056	0.147 ± 0.029	0.736 ± 0.020	0.794 ± 0.028	19.5 ± 4.1	6.3	3.6 ± 0.9
G2 fulldata	0.890 ± 0.009	-0.016 ± 0.009	0.719 ± 0.039	0.126 ± 0.020	0.736 ± 0.018	0.801 ± 0.022	18.3 ± 2.5	8.1	3.0 ± 1.2
G3 balanced	0.849 ± 0.005	0.031 ± 0.018	0.824 ± 0.006	0.045 ± 0.002	0.721 ± 0.004	0.836 ± 0.004	5.7 ± 1.9	1.4	—
G4 fulldata balanced	0.869 ± 0.002	-0.014 ± 0.013	0.862 ± 0.002	0.030 ± 0.007	0.710 ± 0.003	0.852 ± 0.002	1.9 ± 1.3	3.2	—
NCL reference	0.770 ± 0.012	0.057 ± 0.013	0.809 ± 0.022	0.109 ± 0.023	0.639 ± 0.012	0.762 ± 0.011	7.2 ± 1.0	0.3	125.0
<i>Momentum 0.9</i>									
G1 vanilla	0.928 ± 0.003	0.058 ± 0.007	0.797 ± 0.028	0.092 ± 0.016	0.808 ± 0.012	0.878 ± 0.014	15.9 ± 2.6	1.0	12.6 ± 1.7
G2 fulldata	0.964 ± 0.002	-0.010 ± 0.003	0.807 ± 0.009	0.082 ± 0.004	0.808 ± 0.012	0.885 ± 0.004	14.9 ± 0.9	3.1	10.6 ± 1.1
G3 balanced	0.932 ± 0.005	0.038 ± 0.003	0.897 ± 0.006	0.040 ± 0.003	0.822 ± 0.011	0.922 ± 0.003	5.9 ± 0.6	0.4	—
G4 fulldata balanced	0.967 ± 0.001	-0.022 ± 0.003	0.942 ± 0.005	0.016 ± 0.004	0.834 ± 0.010	0.950 ± 0.002	1.3 ± 0.4	0.2	—
NCL reference	0.810 ± 0.019	0.021 ± 0.004	0.871 ± 0.012	0.218 ± 0.014	0.686 ± 0.012	0.778 ± 0.022	8.5 ± 1.1	0.6	5.0

Table C.1: rot-MNIST decomposition block, full metrics, at both momentum settings. Buffer-fidelity diagnostics for these runs are in Table C.3.

C.2 rot-MNIST: λ -Curriculum (C-series)

Condition	ACC	FORG	min-ACC	WF ₁₀₀	WP ₁₀₀	WC-ACC	Depth	Area	Recov.
<i>Momentum 0.0</i>									
C1 linear N50	0.851 $\pm$ 0.023	0.042 $\pm$ 0.021	0.707 $\pm$ 0.019	0.148 $\pm$ 0.011	0.748 $\pm$ 0.006	0.784 $\pm$ 0.021	17.5 $\pm$ 1.7	4.9	35.8 $\pm$ 6.3
C2 linear N100	0.844 $\pm$ 0.025	0.046 $\pm$ 0.023	0.719 $\pm$ 0.033	0.140 $\pm$ 0.025	0.730 $\pm$ 0.006	0.786 $\pm$ 0.019	16.2 $\pm$ 4.3	3.6	64.0 $\pm$ 8.6
C3 linear N200	0.828 $\pm$ 0.025	0.054 $\pm$ 0.024	0.739 $\pm$ 0.048	0.128 $\pm$ 0.043	0.688 $\pm$ 0.004	0.784 $\pm$ 0.034	14.3 $\pm$ 4.6	1.8	132.8 $\pm$ 43.4
C4 adaptive	0.833 $\pm$ 0.025	0.050 $\pm$ 0.019	0.754 $\pm$ 0.025	0.111 $\pm$ 0.016	0.703 $\pm$ 0.007	0.794 $\pm$ 0.020	12.8 $\pm$ 2.9	2.2	88.8 $\pm$ 6.1
C5 adaptive lmin0.05	0.835 $\pm$ 0.025	0.048 $\pm$ 0.019	0.749 $\pm$ 0.027	0.113 $\pm$ 0.016	0.702 $\pm$ 0.006	0.793 $\pm$ 0.020	13.2 $\pm$ 3.2	2.2	88.8 $\pm$ 6.1
C6 adaptive lmin0.10	0.836 $\pm$ 0.026	0.048 $\pm$ 0.021	0.743 $\pm$ 0.026	0.120 $\pm$ 0.019	0.700 $\pm$ 0.007	0.791 $\pm$ 0.019	13.9 $\pm$ 3.3	2.4	88.4 $\pm$ 11.0
C7 adaptive lmin0.20	0.837 $\pm$ 0.031	0.049 $\pm$ 0.024	0.743 $\pm$ 0.030	0.119 $\pm$ 0.023	0.710 $\pm$ 0.010	0.792 $\pm$ 0.024	13.9 $\pm$ 3.4	2.7	64.8 $\pm$ 8.2
<i>Momentum 0.9</i>									
C1 linear N50	0.932 $\pm$ 0.003	0.053 $\pm$ 0.005	0.892 $\pm$ 0.004	0.041 $\pm$ 0.004	0.830 $\pm$ 0.012	0.927 $\pm$ 0.003	6.4 $\pm$ 0.5	0.2	—
C2 linear N100	0.932 $\pm$ 0.003	0.051 $\pm$ 0.008	0.892 $\pm$ 0.005	0.036 $\pm$ 0.003	0.826 $\pm$ 0.012	0.926 $\pm$ 0.002	6.4 $\pm$ 0.6	0.2	—
C3 linear N200	0.925 $\pm$ 0.007	0.051 $\pm$ 0.006	0.893 $\pm$ 0.004	0.031 $\pm$ 0.004	0.819 $\pm$ 0.010	0.920 $\pm$ 0.006	6.2 $\pm$ 0.6	0.1	—
C4 adaptive	0.917 $\pm$ 0.001	0.019 $\pm$ 0.004	0.931 $\pm$ 0.005	0.016 $\pm$ 0.003	0.798 $\pm$ 0.010	0.915 $\pm$ 0.001	2.5 $\pm$ 0.5	0.3	—
C5 adaptive lmin0.05	0.922 $\pm$ 0.001	0.023 $\pm$ 0.004	0.930 $\pm$ 0.005	0.016 $\pm$ 0.002	0.802 $\pm$ 0.009	0.921 $\pm$ 0.001	2.6 $\pm$ 0.5	0.1	—
C6 adaptive lmin0.10	0.927 $\pm$ 0.002	0.028 $\pm$ 0.005	0.926 $\pm$ 0.004	0.017 $\pm$ 0.003	0.806 $\pm$ 0.009	0.927 $\pm$ 0.002	3.0 $\pm$ 0.4	0.0	—
C7 adaptive lmin0.20	0.931 $\pm$ 0.002	0.035 $\pm$ 0.005	0.919 $\pm$ 0.003	0.021 $\pm$ 0.004	0.812 $\pm$ 0.009	0.930 $\pm$ 0.001	3.7 $\pm$ 0.4	0.0	—

Table C.2: rot-MNIST λ -curriculum sweep, full metrics, at both momentum settings. All conditions applied on top of standard ER (1 k reservoir).

C.3 rot-MNIST: Buffer-Fidelity Diagnostics

The gradient diagnostics of Section 6.1.3 were logged for the decomposition block only. The full-data conditions (G2, G4) reach exact fidelity by construction — the replay gradient equals the deterministic empirical past-task gradient, so cosine and magnitude ratio are 1.000. The finite-buffer conditions (G1, G3) show the directional and magnitude error a 1,000-sample reservoir incurs against g_{true} .

Condition	cōs	cos _{min}	$\bar{r}$
G1 vanilla ($\mu=0$)	0.711 $\pm$ 0.035	0.086 $\pm$ 0.103	1.040 $\pm$ 0.039
G3 balanced ($\mu=0$)	0.665 $\pm$ 0.043	0.025 $\pm$ 0.140	1.137 $\pm$ 0.060
G1 vanilla ($\mu=0.9$)	0.551 $\pm$ 0.033	0.135 $\pm$ 0.069	0.418 $\pm$ 0.014
G3 balanced ($\mu=0.9$)	0.441 $\pm$ 0.039	-0.052 $\pm$ 0.117	0.265 $\pm$ 0.024
G2, G4 (full data)	1.000	1.000	1.000

Table C.3: Buffer-fidelity diagnostics for the decomposition block: cōs and cos_{min} are the mean and minimum of $\cos(g_{\text{replay}}, g_{\text{true}})$ over the window; $\bar{r}$ is the mean of $\|g_{\text{replay}}\|/\|g_{\text{true}}\|$.

C.4 CIFAR-10: Headline Block

Condition	ACC	FORG	min-ACC	WF ₁₀₀	WP ₁₀₀	WC-ACC	Depth	Area
D1 vanilla BN	0.723 $\pm$ 0.008	0.010 $\pm$ 0.019	0.644 $\pm$ 0.023	0.127 $\pm$ 0.015	0.484 $\pm$ 0.020	0.667 $\pm$ 0.016	11.8 $\pm$ 2.5	17.5
D2 NCL BN	0.710 $\pm$ 0.005	0.064 $\pm$ 0.026	0.504 $\pm$ 0.045	0.167 $\pm$ 0.031	0.513 $\pm$ 0.018	0.611 $\pm$ 0.020	25.8 $\pm$ 5.2	55.1
D3 adaptive BN	0.722 $\pm$ 0.011	0.006 $\pm$ 0.016	0.713 $\pm$ 0.017	0.070 $\pm$ 0.010	0.469 $\pm$ 0.016	0.698 $\pm$ 0.013	4.8 $\pm$ 0.7	3.6
D4 vanilla GN	0.730 $\pm$ 0.011	-0.023 $\pm$ 0.030	0.309 $\pm$ 0.111	0.294 $\pm$ 0.097	0.415 $\pm$ 0.058	0.507 $\pm$ 0.054	41.2 $\pm$ 13.5	34.9
D5 NCL GN	0.711 $\pm$ 0.017	0.012 $\pm$ 0.028	0.423 $\pm$ 0.083	0.200 $\pm$ 0.062	0.391 $\pm$ 0.055	0.563 $\pm$ 0.038	29.7 $\pm$ 10.0	52.6
D6 adaptive GN	0.731 $\pm$ 0.020	-0.026 $\pm$ 0.026	0.670 $\pm$ 0.055	0.095 $\pm$ 0.033	0.340 $\pm$ 0.052	0.687 $\pm$ 0.037	5.2 $\pm$ 2.1	1.7

Table C.4: CIFAR-10 headline block, full metrics (ResNet-18, momentum 0.9, buffer 5,000, five seeds). “adaptive” is the shipped curriculum with $\lambda_{\text{min}} = 0.20$.

C.5 CIFAR-10: Generalisation Block

Each shift pairs the shipped curriculum (“ship”: adaptive $\lambda_{\text{min}} = 0.20$, buffer 5,000) against a vanilla-ER control on the same shift, at the indicated normalisation, over three seeds.

Condition	ACC	FORG	min-ACC	WC-ACC	Depth	Area
<i>BatchNorm</i>						
shot noise — vanilla	0.724 ± 0.011	0.017 ± 0.010	0.633 ± 0.017	0.663 ± 0.014	13.1 ± 3.0	21.2
shot noise — ship	0.723 ± 0.014	0.010 ± 0.010	0.708 ± 0.032	0.697 ± 0.016	5.6 ± 1.0	5.8
contrast — vanilla	0.671 ± 0.026	-0.008 ± 0.006	0.610 ± 0.063	0.586 ± 0.048	15.4 ± 3.9	33.4
contrast — ship	0.671 ± 0.044	-0.012 ± 0.007	0.727 ± 0.034	0.643 ± 0.048	3.7 ± 1.1	2.2
3-task — vanilla	0.740 ± 0.006	-0.011 ± 0.013	0.670 ± 0.015	0.691 ± 0.010	3.3 ± 0.4	12.0
3-task — ship	0.745 ± 0.010	-0.021 ± 0.008	0.677 ± 0.025	0.696 ± 0.017	6.9 ± 2.7	4.1
<i>GroupNorm</i>						
shot noise — vanilla	0.718 ± 0.021	-0.036 ± 0.036	0.244 ± 0.075	0.469 ± 0.032	46.3 ± 12.0	54.4
shot noise — ship	0.705 ± 0.061	-0.028 ± 0.012	0.638 ± 0.108	0.656 ± 0.087	7.3 ± 5.9	2.4
contrast — vanilla	0.777 ± 0.022	-0.067 ± 0.021	0.641 ± 0.061	0.710 ± 0.040	8.4 ± 4.4	3.3
contrast — ship	0.778 ± 0.032	-0.070 ± 0.013	0.668 ± 0.077	0.724 ± 0.054	3.9 ± 3.1	1.2
3-task — vanilla	0.722 ± 0.020	-0.023 ± 0.015	0.468 ± 0.043	0.550 ± 0.023	4.4 ± 0.4	24.8
3-task — ship	0.721 ± 0.019	-0.025 ± 0.020	0.421 ± 0.114	0.517 ± 0.070	32.8 ± 14.5	16.9

Table C.5: CIFAR-10 generalisation block, full metrics. The two-task novel corruptions (shot noise, contrast) favour the curriculum under both normalisations; the three-task group-norm sequence is the outlier discussed in Section 7.4.

C.6 Long-Sequence rot-MNIST (L-series)

Condition	ACC	FORG	min-ACC	WF ₁₀₀	WP ₁₀₀	WC-ACC	Depth	Area
L1 vanilla ER	0.889 ± 0.007	0.096 ± 0.008	0.845 ± 0.007	0.065 ± 0.008	0.317 ± 0.006	0.870 ± 0.006	8.9 ± 1.7	1.0
L2 NCL	0.815 ± 0.003	-0.033 ± 0.004	0.811 ± 0.006	0.159 ± 0.009	0.423 ± 0.006	0.783 ± 0.006	2.5 ± 0.6	0.4
L3 adaptive lmin0.00	0.896 ± 0.003	0.037 ± 0.003	0.877 ± 0.003	0.030 ± 0.001	0.426 ± 0.005	0.883 ± 0.003	4.4 ± 0.2	1.5
L4 adaptive lmin0.10	0.896 ± 0.003	0.060 ± 0.003	0.877 ± 0.006	0.034 ± 0.004	0.384 ± 0.005	0.890 ± 0.005	4.5 ± 0.6	0.3

Table C.6: Five-task rot-MNIST (rotations $[0, 30, 60, 90, 120]^\circ$, four transitions, momentum 0.9, five seeds). Depth and area are means over the four task boundaries.

D Detailed Step-by-Step Proof of the Trajectory Bound

The derivation below is set in a single column because several of its intermediate expressions are wider than a single text column; the result is the $O(1/N)$ trajectory bound stated in Section 4.

The Mathematical Setup & Definitions

1. The Combined Loss. The total loss function at any point $\lambda \in [0, 1]$ in the curriculum is defined as:

$$L_\lambda(\theta) = L_{\text{replay}}(\theta) + \lambda \cdot L_{\text{current}}(\theta) \quad (\text{D.1})$$

2. Defining the Hessians (∇^2). When we take the derivative of a gradient (∇), we obtain the second derivative matrix, called the Hessian (H).

- $H_{\text{replay}} = \nabla^2 L_{\text{replay}}(\theta)$ (The curvature of the old task)
- $H_{\text{current}} = \nabla^2 L_{\text{current}}(\theta)$ (The curvature of the new task)

Therefore, the Hessian of our combined loss, H_λ , is defined by applying the second derivative operator to the total loss function:

$$\begin{aligned} H_\lambda &= \nabla^2 [L_{\text{replay}}(\theta) + \lambda \cdot L_{\text{current}}(\theta)] \\ H_\lambda &= \nabla^2 L_{\text{replay}}(\theta) + \lambda \cdot \nabla^2 L_{\text{current}}(\theta) \\ H_\lambda &= H_{\text{replay}} + \lambda \cdot H_{\text{current}} \end{aligned}$$

3. The Local-Minimum Assumption. The derivation does not require the loss to be a single global convex bowl. Positive-definiteness of H_λ is used in only three places — inverting H_λ to obtain the path velocity (Part A, Step 2), fixing the sign of the quadratic form (Part A, Step 3), and contracting the tracking error (Part B, Step 3) — and each is a property of the single point $\Theta^*(\lambda)$ on the followed path. We therefore replace the global assumption with the local *second-order sufficient condition along the branch*:

$L_{\text{replay}}, L_{\text{current}} \in C^2$ in a neighbourhood of the path; there is a C^1 branch $\lambda \mapsto \Theta^*(\lambda)$ of critical points of L_λ with $\Theta^*(0) = \theta_0^*$, along which $H_\lambda(\Theta^*(\lambda)) \succ 0$ for every $\lambda \in [0, 1]$.

This is the statement that the iterate tracks a continuously connected family of *strict local* minima, strictly weaker than global convexity. Near $\lambda = 0$ it holds for free: θ_0^* being a non-degenerate local minimum of L_{replay} (ordinary convergence on T_0) gives $H_0 = H_{\text{replay}}(\theta_0^*) \succ 0$, the implicit function theorem then produces a unique smooth branch in a neighbourhood of $\lambda = 0$, and Weyl’s inequality on $H_\lambda = H_{\text{replay}} + \lambda H_{\text{current}}$ bounds how far it persists:

$$\lambda_{\min}(H_\lambda) \geq \lambda_{\min}(H_{\text{replay}}) + \lambda \lambda_{\min}(H_{\text{current}}).$$

If the new task is also locally convex at the relevant points ($H_{\text{current}} \succeq 0$), positive-definiteness holds for all $\lambda \geq 0$ with no global assumption. If H_{current} carries negative curvature ($\mu_- := \lambda_{\min}(H_{\text{current}}) < 0$), it is guaranteed at least while

$$\lambda < \lambda^* = \frac{\lambda_{\min}(H_{\text{replay}})}{-\lambda_{\min}(H_{\text{current}})}. \quad (\text{D.2})$$

Beyond λ^* the branch can reach a λ where H_λ turns singular — a fold bifurcation, where the path velocity $\frac{d\Theta^*}{d\lambda} = -H_\lambda^{-1} \nabla L_{\text{current}}$ diverges, the optimum path jumps discontinuously, and the no-overshoot guarantee genuinely fails. This is the regime, flagged for the deeper CIFAR backbone in Section 6.2.4, where the new task’s negative curvature overpowers the old task’s least curvature and the bound is expected to weaken.

Part A: The Perfect Path Never Spikes

Let $\Theta^*(\lambda)$ be the “Optimum Path”—the exact parameter state that perfectly minimises $L_\lambda(\theta)$ for any given λ .

Step 1: The First-Order Condition. Because $\Theta^*(\lambda)$ is the perfect minimum, the gradient (slope) of the total loss evaluated at that exact point must be zero. We define this as our starting equilibrium:

$$\begin{aligned}\nabla [L_\lambda(\Theta^*(\lambda))] &= 0 \\ \nabla [L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot L_{\text{current}}(\Theta^*(\lambda))] &= 0\end{aligned}$$

Distributing the gradient operator to both terms gives us the fundamental First-Order Condition:

$$\nabla L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) = 0 \quad (\text{D.3})$$

Step 2: Implicit Differentiation to Find Path Movement. We want to know how the optimal path Θ^* moves as λ changes. We apply the derivative operator $\frac{d}{d\lambda}$ to the entire First-Order Condition:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda)) + \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = \frac{d}{d\lambda} [0]$$

Distributing the operator to each term:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda))] + \frac{d}{d\lambda} [\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = 0$$

Evaluating the first term via the multivariable chain rule:

$$\frac{d}{d\lambda} [\nabla L_{\text{replay}}(\Theta^*(\lambda))] = \nabla^2 L_{\text{replay}}(\Theta^*(\lambda)) \cdot \frac{d\Theta^*}{d\lambda}$$

Evaluating the second term via the product rule and chain rule:

$$\frac{d}{d\lambda} [\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))] = (1) \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) + \lambda \cdot \left(\nabla^2 L_{\text{current}}(\Theta^*(\lambda)) \cdot \frac{d\Theta^*}{d\lambda} \right)$$

Substituting these expanded terms back into the equation:

$$\left[\nabla^2 L_{\text{replay}}(\Theta^*) \cdot \frac{d\Theta^*}{d\lambda} \right] + \left[\nabla L_{\text{current}}(\Theta^*) + \lambda \cdot \nabla^2 L_{\text{current}}(\Theta^*) \cdot \frac{d\Theta^*}{d\lambda} \right] = 0$$

Grouping the terms that share the path movement $\frac{d\Theta^*}{d\lambda}$:

$$(\nabla^2 L_{\text{replay}}(\Theta^*) + \lambda \cdot \nabla^2 L_{\text{current}}(\Theta^*)) \cdot \frac{d\Theta^*}{d\lambda} + \nabla L_{\text{current}}(\Theta^*) = 0$$

Recognising the definition of the combined Hessian H_λ inside the parentheses:

$$H_\lambda \cdot \frac{d\Theta^*}{d\lambda} + \nabla L_{\text{current}}(\Theta^*(\lambda)) = 0$$

Solving algebraically for the path movement $\frac{d\Theta^*}{d\lambda}$:

$$\frac{d\Theta^*}{d\lambda} = -H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) \quad (\text{D.4})$$

Step 3: Calculating the Change in Task 0 Loss. How does the Replay loss specifically change as λ increases along this perfect path? We apply the derivative operator:

$$\frac{d}{d\lambda} [L_{\text{replay}}(\Theta^*(\lambda))] = \nabla L_{\text{replay}}(\Theta^*(\lambda))^\top \cdot \frac{d\Theta^*(\lambda)}{d\lambda}$$

We perform two substitutions here. First, from (D.3), we know $\nabla L_{\text{replay}} = -\lambda \cdot \nabla L_{\text{current}}$. Second, from (D.4), we substitute the path movement.

$$\begin{aligned} \frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) &= (-\lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)))^\top \cdot (-H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))) \\ &= \lambda \cdot \nabla L_{\text{current}}(\Theta^*(\lambda))^\top \cdot H_\lambda^{-1} \cdot \nabla L_{\text{current}}(\Theta^*(\lambda)) \end{aligned}$$

Because H_λ is positive-definite along the followed branch of local minima (the Local-Minimum Assumption, valid at least for $\lambda < \lambda^*$ of Equation (D.2)), H_λ^{-1} is positive-definite. The quadratic form $v^\top M v$ is therefore guaranteed to be ≥ 0 . Since $\lambda \geq 0$, the entire equation satisfies:

$$\frac{d}{d\lambda} L_{\text{replay}}(\Theta^*(\lambda)) \geq 0$$

Conclusion for Part A: The Replay loss mathematically never decreases along the perfect path, meaning it can never overshoot its final joint-optimum value.

Part B: The Clumsy Network Tracks the Perfect Path

Let $\theta(t)$ be the actual position of the model parameters at time t . We define the tracking error $\delta(t)$:

$$\delta(t) = \theta(t) - \Theta^*(\lambda(t))$$

Step 1: Differentiating the Error. We apply the time derivative operator $\frac{d}{dt}$ to our error definition:

$$\frac{d}{dt} [\delta(t)] = \frac{d}{dt} [\theta(t)] - \frac{d}{dt} [\Theta^*(\lambda(t))]$$

The model moves via gradient descent, so $\frac{d\theta}{dt} = -\nabla L_{\lambda(t)}(\theta(t))$. The perfect path moves via the chain rule $\frac{d\Theta^*}{dt} = \frac{d\Theta^*}{d\lambda} \cdot \frac{d\lambda}{dt}$. Because the curriculum has a constant velocity over N steps, $\frac{d\lambda}{dt} = \frac{1}{N}$.

$$\frac{d\delta}{dt} = -\nabla L_{\lambda(t)}(\theta(t)) - \left(\frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \right)$$

Step 2: Taylor Expansion of the Actual Gradient. We approximate the gradient at the actual position $\theta(t)$ by Taylor-expanding it around the perfect position Θ^* :

$$\nabla L_\lambda(\theta) \approx \nabla L_\lambda(\Theta^*) + \nabla [\nabla L_\lambda(\Theta^*)] \cdot (\theta - \Theta^*)$$

Since the gradient at the perfect minimum is zero ($\nabla L_\lambda(\Theta^*) = 0$) and the derivative of the gradient is the Hessian ($\nabla^2 L_\lambda = H_\lambda$), this collapses to:

$$\nabla L_\lambda(\theta) \approx H_\lambda \cdot \delta$$

Step 3: The Final Steady State. Substitute this expanded gradient back into our differential equation:

$$\frac{d\delta}{dt} \approx -H_\lambda \cdot \delta - \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N}$$

At a steady-state tracking lag, the error stops growing ($\frac{d\delta}{dt} \approx 0$):

$$\begin{aligned} 0 &\approx -H_\lambda \cdot \delta - \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \\ H_\lambda \cdot \delta &\approx -\frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \\ \delta &\approx -H_\lambda^{-1} \cdot \frac{d\Theta^*}{d\lambda} \cdot \frac{1}{N} \end{aligned}$$

Taking the norm, we see the lag distance is mathematically bounded by the curriculum length N :

$$\|\delta\| = O\left(\frac{1}{N}\right) \quad (\text{D.5})$$

The Grand Finale: Expanding the Actual Loss

To find the maximum Replay loss the model will actually experience, we Taylor expand the loss of the actual trajectory around the perfect position:

$$\begin{aligned} L_{\text{replay}}(\theta) &\approx L_{\text{replay}}(\Theta^*) + \nabla L_{\text{replay}}(\Theta^*)^\top \cdot (\theta - \Theta^*) \\ L_{\text{replay}}(\theta) &\approx L_{\text{replay}}(\Theta^*) + \nabla L_{\text{replay}}(\Theta^*)^\top \cdot \delta \end{aligned}$$

From Part A, $L_{\text{replay}}(\Theta^*)$ cannot exceed the final joint optimum loss $L_{\text{replay}}(\theta_{\text{joint}}^*)$. From Part B, the lag δ is $O(1/N)$. Substituting these limits yields the final mathematical guarantee:

$$\sup_t L_{\text{replay}}(\theta(t)) \leq L_{\text{replay}}(\theta_{\text{joint}}^*) + O\left(\frac{1}{N}\right) \quad (\text{D.6})$$

Conclusion: The transient stability gap is entirely contained within the $O(1/N)$ term, meaning a sufficiently slow continuous curriculum eliminates the discrete-time overshoot.

Why the Gap Is Large for Small N : Two Mechanisms

The bound above explains why the gap *vanishes* as $N \rightarrow \infty$. The complementary question — why a small N (in the limit, the instantaneous jump $N = 1$) produces a large gap — has a two-part answer, and the two parts map onto the two empirical readouts: the instantaneous **gap_depth** and the sustained **gap_area/recovery_steps**.

Write $g := \nabla L_{\text{current}}(\theta_0^*)$ for the new-task gradient at the boundary (large, because T_1 is unfit), and use convergence on the old task, $\nabla L_{\text{replay}}(\theta_0^*) = 0$ and $H_0 = H_{\text{replay}}(\theta_0^*) \succ 0$. For a linear ramp $\lambda_k = k/N$ the first applied weight is $\lambda_1 = 1/N$. Linearising the iterate error $e_k = \theta_k - \Theta^*(\lambda_k)$ around the path gives the forced recursion

$$e_{k+1} = (I - \eta H_{\lambda_k}) e_k + r_k, \quad (\text{D.7})$$

whose two contributions — the initial transient and the particular solution under the forcing r_k — are the two mechanisms.

Mechanism 1: the first-step magnitude overshoot (depth). Because $\nabla L_{\text{replay}}(\theta_0^*) = 0$, the first update is *one-sided*: the replay gradient contributes nothing and the step is driven entirely by the new task,

$$\theta_1 - \theta_0^* = -\eta \lambda_1 g,$$

which is precisely the magnitude asymmetry G_{mag} of Section 3.1. Any displacement d away from the replay minimum raises the replay loss purely through curvature, since the first-order term vanishes at θ_0^* :

$$L_{\text{replay}}(\theta_1) - L_{\text{replay}}(\theta_0^*) \approx \frac{1}{2} d^\top H_{\text{replay}} d = \frac{1}{2} \eta^2 \lambda_1^2 g^\top H_0 g = O(\lambda_1^2). \quad (\text{D.8})$$

This is the worst single point of instability — **gap_depth**. It scales as λ_1^2 : $O(1)$ for the jump ($\lambda_1 = 1$) and $O(1/N^2)$ for a ramp. The spike then closes geometrically as the homogeneous part of Equation (D.7) pulls the iterate back into the basin, $(I - \eta H_0)^k$, at rate $\eta \lambda_{\min}(H_0)$ — the recovery. No path-tracking argument enters here; the depth is first step $\times$ curvature.

Mechanism 2: the trajectory lag (the hanging tail). Even with the first-step kick removed, the iterate cannot sit on the *moving* minimiser. The particular solution of Equation (D.7) is the sustained lag of Part B,

$$\delta_{\text{ss}} = -H_\lambda^{-1} \frac{d\Theta^*}{d\lambda} \frac{1}{N} = O\left(\frac{1}{N}\right),$$

held across the *entire* ramp rather than at one step. It is therefore a broad, slowly-closing deviation — the dip below the recovered level measured by **gap_area**, and the delay measured by **recovery_steps** — the hanging tail rather than the notch.

The combined small- N picture. Both forcings are governed by the same N : the trajectory forcing is $\propto 1/N$, and the first-step weight is $\lambda_1 = 1/N$. At the instantaneous jump ($N = 1$) the optimum-path velocity is effectively infinite and the boundary kick is full strength, so both mechanisms fire at $O(1)$ and the gap is maximal. Growing N attenuates them at *different* rates — $O(1/N^2)$ for the depth spike (Equation (D.8)) versus $O(1/N)$ for the tail — which is why the trajectory term is the bottleneck in the combined bound of Equation (4.3) and why the curriculum tightens **gap_area** more cleanly than **gap_depth**.

Reconciliation with the measured depth. Equation (D.8) predicts the *schedule-dependent* part of depth should fall as $1/N^2$. The measured depth (conditions C1–C3) only drifts from ≈ 17.5 to ≈ 14.3 rather than vanishing, because what remains is the estimator-variance floor G_{est} — a property of the finite buffer, untouched by $\lambda(t)$ — which the decomposition isolates as the residual depth at G4. The theory thus predicts the controllable component of depth, and the observed floor is exactly the schedule-independent contributor.